M03 — Prácticas de Blazor Web App y componentes

Desarrollo Web con ASP.NET Core MVC, Razor y Blazor en .NET 8

CURSO: Desarrollo Web con ASP.NET Core MVC, Razor y Blazor en .NET 8

TIPO: PRÁCTICAS

AUTOR: JAIME GALLO

Itinerario de trabajo y condiciones del laboratorio

Durante esta jornada utilizaremos una única aplicación B de Blazor Web App en .NET 8, organizada en seis **estados reproducibles** dentro de la carpeta `M03_BLAZOR/PROYECTO/`. Cada punto tiene un proyecto `CursoWeb.Blazor` independiente que conserva la misma estructura general. Eso permite repetir un experimento desde su estado inicial sin desmontar el código de prácticas anteriores; no representa seis aplicaciones de negocio distintas. El alumno conoce C# y las bases de Web API, pero no se presupone que conozca Blazor. No se requiere SQL Server, LocalDB ni API remota para M03.

Los proyectos están preparados con comentarios pedagógicos identificados por paso. En archivos C#, se activa el bloque delimitado por `/* PASO X INICIO` y `PASO X FIN */` retirando **únicamente los delimitadores**. En archivos Razor se retiran `@*` y `*@` del fragmento correspondiente. Cuando se proponen variantes completas de un componente, hay que **sustituir** una por otra, no dejar varias instancias visibles simultáneamente. El alumno no tiene que adivinar qué líneas aparecieron entre un estado y el siguiente.

Antes de cada práctica detén el proceso de la anterior y abre el `CursoWeb.Blazor.csproj` de la carpeta concreta. Para P3.1, por ejemplo:

```
cd M03_BLAZOR/PROYECTO/3.1/CursoWeb.Blazor
dotnet restore
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5092
```

Abre después `http://localhost:5092`. Si Visual Studio utiliza un puerto distinto, sigue la dirección anunciada por Kestrel. El proyecto debe ejecutarse con el SDK de .NET 8. No copies los cambios en M01 ni M02; son aplicaciones y materiales anteriores. Los pasos esenciales forman la práctica de clase; los experimentos de transferencia y retos amplían la comprensión sin requerir una infraestructura adicional.

Índice del cuaderno

- P3.1. Comparación de SSR estático e Interactive Server.
- P3.2. Página Productos y navegación con NavLink.
- P3.3. Parámetros y componente TarjetaProducto.
- P3.4. Estado, eventos y binding.
- P3.5. Comunicación padre–hijo con EventCallback.
- P3.6. Servicio asíncrono, OnInitializedAsync y estados de carga.

P3.1. Crear la aplicación Blazor B y comprobar interactividad real

Objetivo y estado inicial

El objetivo es identificar el problema que resuelve Interactive Server y comprobarlo empíricamente. La carpeta inicial es `M03_BLAZOR/PROYECTO/3.1/CursoWeb.Blazor`. Dispone de dos páginas equivalentes en lo esencial: `Estatica.razor` sin modo interactivo y `Interactiva.razor` con ese modo declarado. El primero permite observar que escribir un manejador de clic no es suficiente; el segundo comprueba que el mismo tipo de manejador sí funciona cuando la aplicación está configurada correctamente.

3.1.1. Abrir y compilar el estado de partida

Abre exactamente `M03_BLAZOR/PROYECTO/3.1/CursoWeb.Blazor/CursoWeb.Blazor.csproj`. Comprueba que el archivo de proyecto declara `net8.0`. En una consola situada en esa carpeta, ejecuta `dotnet restore` y `dotnet build`. Si alguna operación falla, verifica la versión del SDK y que abriste este proyecto, no uno de los módulos MVC.

Ejecuta con `dotnet run --no-launch-profile --urls http://localhost:5092`. El resultado esperado es que Kestrel anuncie la URL. Abre el navegador y visita la raíz. Debes encontrar el título «Aplicación B · Blazor Web App», una descripción del catálogo en memoria y enlaces a las dos formas de renderizado.

Relación con la teoría: 3.1 explica por qué Blazor sigue ejecutándose dentro del host ASP.NET Core aunque cambie el modelo de interacción. En `Program.cs` verifica las dos instrucciones que preparan el servidor:

```
builder.Services.AddRazorComponents().AddInteractiveServerComponents();
app.MapRazorComponents<App>().AddInteractiveServerRenderMode();
```

La primera configura servicios; la segunda expone endpoints de componentes con soporte de interactividad en servidor. Si falta una parte,

no debemos concluir que un método `@onclick` funcionará solo porque el proyecto contiene código C#.

3.1.2. Identificar las piezas del proyecto

Abre `Components/App.razor` y localiza el documento HTML, `<Routes />` y el script `_framework/blazor.web.js`. Abre `Components/Routes.razor` y encuentra `Router` y `RouteView`. Abre `Components/Layout/MainLayout.razor` y localiza `@Body`. Comprueba que `Components/Pages/Home.razor` declara `@page "/"`.

Explica en tu cuaderno el recorrido de la ruta inicial: el navegador solicita una URL; ASP.NET Core atiende el endpoint; el router identifica la página; el layout la representa dentro de una estructura común. Esta explicación se parece conceptualmente al recorrido MVC que ya conoces, pero ahora no existe `HomeController` para esta pantalla.

Como caso negativo, abre `/una-ruta-inexistente`. Debes ver el contenido de la rama `NotFound` del router. La aplicación debe continuar respondiendo en `/`. Esta observación ayuda a distinguir una ruta inexistente de una aplicación que no ha arrancado.

3.1.3. Observar un botón bajo SSR estático

Abre `Components/Pages/Estatica.razor`. Comprueba que existe:

```
@page "/estatica"
<p id="contador-estatico">Pulsaciones: @contador</p>
<button id="boton-estatico" @onclick="Incrementar">Pulsar (sin interacción)</
button>

@code {
    private int contador;
    private void Incrementar() => contador++;
}
```

Visita `http://localhost:5092/estatica` y pulsa el botón tres veces. **Resultado esperado:** «Pulsaciones: 0» continúa visible. Esto no constituye un fallo del ejercicio. El componente tiene código y se ha representado desde el

servidor, pero no está habilitado como componente interactivo capaz de procesar ese clic C# después de la respuesta inicial.

No intentes solucionar este resultado sustituyendo el método por JavaScript. El propósito es aislar una condición del modelo de ejecución. En las herramientas de desarrollo observa que pulsar no desencadena una navegación completa con una nueva acción MVC, y describe qué necesitaría el componente para responder.

3.1.4. Observar el mismo evento con Interactive Server

Abre `Components/Pages/Interactiva.razor`. Sus primeras líneas son:

```
@page "/interactiva"
@rendermode InteractiveServer
```

El resto conserva un contador y un botón con manejador de clic. Abre `http://localhost:5092/interactiva`, deja que la conexión se establezca y pulsa una vez. Debes ver «Pulsaciones: 1». Vuelve a pulsar: el número pasa a dos. No se modifica el servidor SQL ni una colección global: el campo `contador` pertenece a la instancia interactiva del componente.

Compara qué cambia en los dos archivos: no necesitas escribir otro lenguaje de programación; el elemento diferenciador es `@rendermode InteractiveServer` y el soporte configurado en `Program.cs`. Revisa también que en `App.razor` el proyecto utiliza `<Routes />` sin hacer interactivo todo el árbol; por eso ambas páginas pueden coexistir con comportamientos distintos.

3.1.5. Diagnóstico y verificación negativa

Retira temporalmente la directiva `@rendermode InteractiveServer` de `Interactiva.razor`, recompila y abre esa página desde cero. El botón deja de incrementar el valor, como en `/estatica`. Restablece la directiva y verifica que vuelve a funcionar. En una explicación de una o dos frases, relaciona ambos resultados con la diferencia entre representación estática y circuito interactivo, sin atribuirlos a la apariencia de los botones.

Si la página interactiva continúa sin responder, comprueba por este orden: ruta correcta, directiva de renderizado, registro de servicios en `Program.cs`, mapeo de endpoints, script en `App.razor` y errores de conexión que muestre el navegador. Un compilador con salida correcta no garantiza por sí solo la interacción de navegador: debes observar un clic que cambie el texto.

Reto 3.1 — Incrementos dobles

Enunciado: modifica una sola línea del método de evento de `Components/Pages/Interactiva.razor` para sumar dos unidades por pulsación, manteniendo intacta `Estatica.razor`. El objetivo es demostrar que el comportamiento depende de la lógica de C# y que la posibilidad de ejecutarla depende del modo de renderizado.

Código de solución para consulta:

```
private void Incrementar() => contador += 2;
```

Reemplaza temporalmente `contador++` por la línea anterior. Compila, navega a `/interactiva` y comprueba los valores 2 y 4 tras dos clics. **Control negativo:** `/estatica` sigue sin variar al pulsar. Restaura después `contador++`. La solución no añade nuevos endpoints, ni un formulario POST, ni persistencia. La modificación consiste únicamente en la operación realizada en el manejador.

Profundización opcional

Explica por qué el primer GET de una página interactiva sigue utilizando HTTP, aunque los eventos posteriores no se gestionen mediante una nueva acción MVC completa. Analiza qué puede ocurrir si el navegador pierde su conexión con el servidor mientras el contador está visible. No necesitas estudiar SignalR avanzado ni capturar los mensajes de protocolo: basta comprender que Interactive Server mantiene lógica y estado en el servidor y depende de esa comunicación.

Cuaderno de profundización P3.1 — Distinguir cinco tipos de evidencia

El itinerario esencial ya ha demostrado que el modo de renderizado determina si la interacción C# está disponible. La siguiente actividad permite profundizar en **cómo sabemos que algo funciona**. No requiere archivos adicionales ni obliga a modificar de forma permanente el proyecto. Realízala después de completar 3.1.5 y antes de pasar a P3.2, o consérvala como ejercicio de consulta.

Primer tipo de evidencia: el host está iniciado

Abre una consola en la carpeta del proyecto 3.1 y ejecuta `dotnet run --no-launch-profile --urls http://localhost:5092`. Observa el mensaje del host que indica que Kestrel está escuchando. Esa salida demuestra que el proceso se ha iniciado, pero **no** demuestra que la página tenga un evento interactivo correcto. Es posible que un servidor responda HTTP y que después un componente falle al renderizar; por eso la siguiente observación debe realizarse desde el navegador.

Visita `/` y comprueba que aparece el texto de la aplicación B. Si recibes «No se puede conectar», revisa la URL y el puerto, y confirma que no estás ejecutando otro proyecto. Si aparece una pantalla con el catálogo MVC de M02, detén el proceso anterior y vuelve a iniciar 3.1. Esta incidencia se resuelve identificando el **host correcto**, no cambiando el código de un contador.

Segundo tipo: una URL corresponde a un componente

Abre `/estatica` y `/interactiva` en pestañas distintas. En cada una observa su encabezado y el código fuente de la página. La primera corresponde a `Estatica.razor` y la segunda a `Interactiva.razor`. Ambas declaran una ruta mediante `@page`. La respuesta del servidor confirma que el router puede seleccionar los componentes; no demuestra todavía que el botón produzca un evento.

Como caso negativo, abre `/no-existe`. El router presenta la rama de página no encontrada. Contrasta este resultado con una aplicación que no arranca: en uno puedes recibir una respuesta HTML de la propia aplicación y en el otro no llegas a establecer comunicación con el host. Escribe dos frases que permitan a otro alumno distinguir los síntomas sin acceder al código.

Tercero: el HTML contiene un botón y un estado inicial

En cada página identifica el párrafo que muestra las pulsaciones. El HTML inicial presenta un valor cero. Eso demuestra que Razor ha podido evaluar el campo `contador` durante la representación. No demuestra que el método de evento pueda ejecutarse **después** de cargar la página. Este detalle es importante porque un test automatizado que solo descargara la dirección inicial no verificaría la interactividad de Blazor.

Abre las herramientas de desarrollo, inspecciona los elementos `#boton-estatico` y `#boton-interactivo` y observa su parecido. Comparten la forma de describir un evento en Razor, pero participan en modos de ejecución diferentes. Un estilo visual parecido tampoco equivale a un comportamiento idéntico; por eso necesitamos el cuarto tipo de evidencia.

Cuarto: el navegador ejecuta una interacción real

Pulsa el botón estático dos veces y observa que el número permanece en cero. Después pulsa el interactivo dos veces y comprueba que aparece dos. Esta es la evidencia que demuestra el objetivo principal. Anota además si la URL de la página cambió durante la operación; el contador se actualiza sin que debamos crear una ruta específica para cada valor.

Si el botón interactivo no responde, no declares éxito basándote en la existencia de `@onclick` en el archivo. Comprueba `@rendermode`, los servicios y el script, y examina la consola del navegador. La prueba positiva exige un cambio visible generado como consecuencia de un clic concreto.

Quinto: continuidad del estado y dependencia de la sesión

Con el contador interactivo en tres, recarga completamente la página. Observa si comienza una nueva instancia que muestra de nuevo el valor inicial. En nuestro laboratorio no hay base de datos ni servicio de persistencia que conserve ese valor. Explica por qué una recarga y un clic son operaciones distintas: una puede crear una nueva instancia de componente, mientras el otro modifica el estado de la instancia activa.

No es necesario medir la duración exacta del circuito ni estudiar procedimientos de reconexión avanzados. El objetivo es distinguir **estado de componente** de **dato persistente** y reconocer que Interactive Server necesita comunicación con el servidor para los eventos.

Resolución de tres incidencias simuladas

Incidencia A. El alumno observa el contador estático en cero y asegura que el método C# es incorrecto. La explicación razonada es que el método puede estar bien escrito, pero la página no declara interactividad; el problema es de **modo de ejecución**, no de aritmética.

Incidencia B. El alumno ve `/interactiva`, pulsa y no ocurre nada tras perder conexión. Debe verificar el circuito y los recursos del navegador antes de cambiar el código de `Incrementar`. No puede garantizar que el evento haya llegado al servidor.

Incidencia C. El alumno pulsa, el número llega a cinco, recarga y vuelve a cero. No se trata de un error de `@bind` ni de un problema de SQL; la variable es un campo local y no tiene almacenamiento permanente.

Elabora una pequeña tabla personal con la causa y la evidencia de cada incidencia. Compara tus respuestas con la teoría 3.1 sobre SSR, Interactive Server y vida del estado. Si sabes explicar por qué una página puede compilar, responder HTTP y aun así no atender un clic C#, has alcanzado una comprensión que se trasladará a todos los puntos siguientes.

Comparación transversal con M02

En el CRUD MVC, una respuesta HTTP correcta después de un POST podía terminar en una redirección a Detalle y una consulta a LocalDB. En el contador Blazor, la acción no necesita navegar ni guardar. No confundas «respuesta del servidor» con «persistencia»: son responsabilidades diferentes. M03 construirá una interfaz interactiva y M04 añadirá transporte remoto; cada fase tendrá pruebas adecuadas a lo que verdaderamente ejecuta.

P3.2. Página Productos y navegación con NavLink

Objetivo y estado inicial

El proyecto `M03_BLAZOR/PROYECTO/3.2/CursoWeb.Blazor` conserva las páginas anteriores y añade el modelo `Producto` y la página `/productos`. La ruta **ya funciona**, pero el enlace del menú permanece comentado para aprender que declarar un endpoint y ofrecer navegación son responsabilidades diferentes.

3.2.1. Abrir el proyecto y probar la ruta directamente

Detén el host de P3.1 y abre `M03_BLAZOR/PROYECTO/3.2/CursoWeb.Blazor/CursoWeb.Blazor.csproj`. Compila y ejecuta en el puerto 5092. Sin utilizar el menú, abre directamente `http://localhost:5092/productos`. Debes encontrar un listado de Cuaderno, Lápiz y Carpeta. Si la dirección funciona pero todavía no existe el enlace «Productos» en la cabecera, ese es exactamente el estado inicial esperado.

Abre `Components/Pages/Productos.razor` y encuentra las directivas `@page "/productos"` y `@rendermode InteractiveServer`. La primera declara la dirección. La segunda anticipa la interactividad que añadiremos después, aunque por ahora solo representemos una lista.

3.2.2. Activar el enlace preparado en el layout

Abre `Components/Layout/MainLayout.razor` y localiza la marca `PASO 3.2.2 (ACTIVAR)`. Justo debajo aparece:

```
@* <NavLink href="/productos">Productos</NavLink> *@
```

Elimina los delimitadores de comentario, conservando:

```
<NavLink href="/productos">Productos</NavLink>
```

Guarda y recompila. Abre `http://localhost:5092/`. El menú debe mostrar «Productos». Haz clic, observa la ruta final y regresa al inicio. El elemento `NavLink` ofrece el enlace y la indicación de navegación activa, pero **no crea por sí mismo la página**; esta ya existía gracias a la directiva `@page`.

3.2.3. Leer el código de modelo y lista

Abre `Models/Producto.cs` y comprueba la declaración del record con Id, Nombre y Precio. En `Components/Pages/Productos.razor` localiza:

```
<ul id="lista-productos">
    @foreach (var producto in productos)
    {
        <li>@producto.Nombre — @producto.Precio.ToString("0.00") €</li>
    }
</ul>
```

Dentro de `@code` se crea una colección con tres elementos. Relaciona la lista C# con los tres elementos `li` del HTML representado. Abre las herramientas del navegador e inspecciona `id="lista-productos"`. No existe ninguna petición SQL ni HTTP a una API para rellenarla: el estado pertenece al componente.

Ejercicio de diagnóstico: si el menú no muestra Productos, investiga el layout; si aparece el menú pero `/productos` no existe, investiga `@page` y el router. No cambies los archivos de M02 para arreglar un problema de esta nueva aplicación.

3.2.4. Comparar tres nombres parecidos que representan ideas distintas

Explica la diferencia entre `Views/Productos/Index.cshtml` de MVC, `Pages/Comparacion.cshtml` de Razor Pages y `Components/Pages/Productos.razor` de Blazor. Los tres generan contenido de interfaz a partir de Razor, pero el primero se elige desde una acción MVC, el segundo usa handlers de PageModel y el tercero es un componente que puede tener renderizado interactivo. Esta comparación enlaza expresamente con los módulos anteriores y evita una confusión que suele aparecer al comenzar con Blazor.

Ejercicio autónomo 3.2 — Añadir una fila controlada

Enunciado: añade un producto «Regla», con Id 4 y precio 2.50, dentro de la colección que declara `Productos.razor`. No dupliques el elemento HTML ni crees otro archivo. **Código que debes escribir:** incorpora un cuarto elemento:

```
new(4, "Regla", 2.50m)
```

Compila y recarga `/productos`. Deben aparecer **cuatro elementos de lista**. El formato del precio debe seguir la expresión `ToString("0.00")`. Si solo ves tres elementos, verifica que editaste el proyecto 3.2 que estás ejecutando. Si el precio se representa con un formato diferente, investiga cómo se convierte el decimal a texto. Después recupera la colección inicial de tres elementos para continuar con P3.3 tal como se entrega.

Profundización opcional

Cambia temporalmente el texto visible del enlace de menú sin modificar su `href`. Observa que la ruta sigue siendo la misma. Ahora imagina que cambiaras `@page "/productos"` a otro nombre sin actualizar el menú: ¿qué enlace quedaría roto? No hace falta alterar definitivamente la ruta; basta explicar que la dirección es un contrato distinto del texto que la representa. Este criterio es útil en MVC, Razor Pages y Blazor.

Cuaderno de profundización P3.2 — Investigar navegación, modelos y representación

Este recorrido opcional se realiza sobre `PROYECTO/3.2/CursoWeb.Blazor` después de activar NavLink. Su objetivo es reconocer qué archivo cambia cada comportamiento de la pantalla. No añade otra página obligatoria ni necesita crear servicios.

Escenario A: ruta correcta, enlace ausente

Abre directamente `/productos`. Si la página funciona, tienes evidencia de que el endpoint existe. Después ve a `/` y comprueba la navegación. Si el enlace

no está, revisa `Components/Layout/MainLayout.razor` y localiza la línea marcada `PASO 3.2.2`. Asegúrate de haber retirado los delimitadores del comentario. No renombres `Productos.razor` ni modifiques su directiva de ruta para solucionar un menú que está comentado.

Este caso muestra una relación de **oferta de navegación**: el usuario puede conocer una URL aunque no aparezca en el menú. Lo mismo ocurría en MVC con las acciones que no tenían enlace visible. La decisión de publicar una ruta y la decisión de mostrarla como opción de interfaz deben estar coordinadas, pero no son la misma operación.

Escenario B: el menú apunta a una página que no existe

Cambia **temporalmente** la dirección del enlace por `/catalogo-inventado`, manteniendo `@page "/productos"` en la página. Compila y haz clic en el menú. Debes observar el comportamiento de ruta no encontrada. El resultado no indica que los tres productos hayan desaparecido de la colección; indica que el navegador fue dirigido a una dirección que el router no puede resolver en este proyecto.

Restablece `href="/productos"` y comprueba que el listado vuelve a estar disponible. La solución razonada es mantener coherencia entre la URL pública y los enlaces; no es crear una segunda lista de productos dentro del layout.

Escenario C: los datos cambian, pero el marco sigue estable

En `Components/Pages/Productos.razor` modifica temporalmente el precio de Cuaderno y recompila. La pantalla mostrará el nuevo valor mientras el menú y la estructura visual permanecen. Esa separación demuestra que `MainLayout.razor` no necesita conocer el precio del producto. Si tu cambio modifica el menú por accidente, has editado el archivo equivocado o mezclado responsabilidades.

Repite con un cambio puramente visual: modifica el texto fijo del encabezado de la página. No necesitas tocar el record Producto para ello. Vuelve después a los textos y precios iniciales.

Escenario D: un cambio de nombre rompe una referencia de C#

Si cambiaras la propiedad `Nombre` del record por otra sin adaptar las expresiones Razor que la consultan, tendrías un problema de compilación o referencias, aunque la ruta continuara siendo correcta. No es necesario ejecutar esa modificación destructiva; basta localizar dónde se utiliza `producto.Nombre` y explicar la dependencia con `Models/Producto.cs`.

La diferencia con los fallos anteriores importa: en A fallaba la navegación visible; en B, el destino de la ruta; en C, solo cambiaba el dato o el texto representado; y en D cambiaría el contrato de C# que el componente utiliza.

Diagnóstico paso a paso del HTML

Inspecciona el elemento `ul` de `/productos`. Debe contener tres `li`. Los elementos son el resultado del bucle Razor, no una respuesta JSON de API. Si la lista aparece vacía, comprueba que los tres objetos siguen definidos en `@code`. Si el HTML contiene el mismo producto varias veces, investiga las entradas de la colección y el cuerpo del bucle.

Observa que `ToString("0.00")` se aplica al representar el precio. La representación cultural puede afectar a separadores decimales según el contexto. El alumno no necesita rediseñar la internacionalización completa del curso, pero debe saber que los datos numéricos y su texto visible son entidades distintas. En M02 ya se examinó la conversión de entrada; aquí hablamos principalmente de formato de salida.

Interpretación del modo interactivo sin necesidad inmediata de eventos

`Productos.razor` declara `@rendermode InteractiveServer` aunque el estado inicial solo represente una lista. Eso no significa que la página ya tenga un botón con

lógica propia. El modo prepara la capacidad interactiva; los eventos específicos se añadirán en P3.4. La diferencia se parece a tener una acción MVC disponible sin que todos los enlaces la invoquen: disponer de infraestructura no determina por sí solo las operaciones concretas que la aplicación realiza.

Solución razonada del ejercicio de transferencia

Si deseas agregar una ruta nueva, crea otro componente de página que declare `@page` y actualiza la navegación cuando tenga sentido. Si quieres cambiar el menú, edita el layout. Si deseas cambiar un producto de esta muestra fija, edita la colección de la página. Si necesitas cambiar el estilo general, edita `wwwroot/app.css`. Y si quieres modificar cómo se representa **cada producto** sin repetir HTML, espera al siguiente punto y utiliza TarjetaProducto.

La finalidad no es aprender una lista de carpetas de memoria. Es descubrir que cada carpeta y archivo responde a una decisión técnica diferente, y que diagnosticar bien ahorra cambios innecesarios.

P3.3. Componente TarjetaProducto, parámetros y composición

Objetivo y preparación

Ahora convertimos una lista HTML sencilla en tres tarjetas generadas mediante un componente reutilizable. El estado inicial de `M03_BLAZOR/PROYECTO/3.3/CursoWeb.Blazor` sigue compilando y mostrando los productos mediante `<ul>`. El proyecto incluye además `Components/TarjetaProducto.razor` y una variante comentada de composición. Necesitamos activar la segunda **sustituyendo** el bloque antiguo para no duplicar la interfaz.

3.3.1. Verificar el estado de salida de P3.2

Detén P3.2, abre el proyecto 3.3, compila y ejecuta. Abre `/productos`. Debes encontrar una lista de tres elementos. El objetivo inmediato no es añadir datos nuevos; es cambiar **cómo** se representan los mismos tres objetos.

En `Components/Pages/Productos.razor` localiza el comentario `PASO 3.3.2` y el bloque activo que comienza con `<ul id="lista-productos">`. Más abajo encuentra `PASO 3.3.3 (ACTIVAR)`, que incluye una variante de tarjetas dentro de un comentario Razor.

3.3.2. Comprender el contrato de la tarjeta

Abre `Components/TarjetaProducto.razor`. Encontrarás un artículo con nombre, precio e identificador:

```
<article class="tarjeta" data-producto-id="@Producto.Id">
    <h2>@Producto.Nombre</h2>
    <p>Precio: @Producto.Precio.ToString("0.00") €</p>
    <small>Referencia @Producto.Id</small>
</article>
```

Debajo aparece:

```
[Parameter, EditorRequired]
public Producto Producto { get; set; } = default!;
```

Explica por qué `[Parameter]` establece un contrato de entrada. La tarjeta no conoce la colección completa, no tiene que consultar una API y no declara `@page`. Solo necesita el producto que recibe del componente padre. La marca `EditorRequired` ayuda a las herramientas a identificar que se espera ese parámetro; no introduce por sí sola un mecanismo de validación HTTP.

3.3.3. Sustituir la lista por tres tarjetas

En `Components/Pages/Productos.razor`, comenta temporalmente el bloque de lista completo, desde `<ul id="lista-productos">` hasta `</ul>`, incluidos los elementos de iteración que contiene. Activa el bloque siguiente retirando `@*` y `*@`:

```
<div class="tarjetas" id="tarjetas-productos">
    @foreach (var producto in productos)
    {
        <TarjetaProducto Producto="producto" />
    }
</div>
```

Compila y abre `/productos`. **Resultado positivo:** tres tarjetas, una por producto, con sus nombres, precios e identificadores. En el inspector del navegador busca `data-producto-id="1"`, `data-producto-id="2"` y `data-producto-id="3"`. El atributo ayuda a comprobar qué producto recibió cada tarjeta.

Caso negativo de transformación: si conservaste activo el bloque `<ul>` además del nuevo bloque, aparecerán dos representaciones del catálogo. Eso no demuestra que `[Parameter]` haya fallado; indica que has dejado dos bucles activos. Deja solo la versión de tarjetas.

3.3.4. Explicar el vínculo entre padre e hijo

En el padre, la variable `producto` procede del `foreach`. En el hijo, la propiedad `Producto` recibe ese objeto. La expresión `Producto="producto"` es el punto donde las dos piezas se encuentran. Si sustituyes temporalmente la variable por

`productos[0]`, las tres tarjetas pueden representar el mismo objeto Cuaderno aunque haya tres iteraciones. Restablece `Producto="producto"` y comprueba que vuelven los tres productos distintos. La prueba demuestra que el componente reutilizable necesita datos correctos del padre; no crea por sí mismo una lista independiente.

3.3.5. Observar un cambio centralizado

En `Components/TarjetaProducto.razor` sustituye temporalmente el texto «Precio:» por «Importe:» y compila. El cambio aparece en las tres tarjetas. No es necesario modificar el padre varias veces. Después restaura el texto original. Esta observación justifica la extracción del componente: una misma responsabilidad de representación está concentrada en un lugar.

Si los elementos se dibujan sin tarjetas visuales, comprueba la clase `tarjeta` y la carga de `wwwroot/app.css`. Si no se muestran los nombres correctos, inspecciona los valores que el padre transmite. No confundas un problema de estilos con un problema de parámetros.

Reto 3.3 — Indicar productos que superan un precio

Enunciado: modifica exclusivamente `Components/TarjetaProducto.razor` para que un producto cuyo precio sea superior a cuatro euros muestre el texto «Precio superior a 4 euros». Se trata de un ejercicio autónomo de representación condicional, no de una nueva regla de negocio o de una llamada a un servicio.

Código de solución razonada:

```
@if (Producto.Precio > 4m)
{
    <p class="mensaje">Precio superior a 4 euros</p>
}
```

Escribe el bloque dentro del artículo, después del precio. Compila y observa el resultado: Cuaderno tiene precio 4.50 y debe mostrar la advertencia; Lápiz y Carpeta no. **Caso negativo:** no deben aparecer

mensajes en productos que cuestan menos de cuatro euros. Para explorar la frontera entre `>` y `>=`, puedes cambiar temporalmente el precio de Carpeta a exactamente 4.00, observar el comportamiento y restaurarlo.

Explicación: esta responsabilidad pertenece al hijo porque es una variación de cómo se representa **un** producto. No hace falta duplicar el bucle en el padre ni crear una vista Razor MVC. El componente recibe el objeto y decide qué HTML adicional producir.

Profundización opcional: reutilización y responsabilidades

Imagina que la página Productos se utiliza más adelante para mostrar resultados de una API. ¿Debería `TarjetaProducto` hacer por sí misma una petición HTTP cada vez que se representa? En nuestro diseño, no: el padre o un servicio coordinador obtiene datos, y las tarjetas reciben los objetos que deben mostrar. Este planteamiento permite reutilizar la representación sin acoplarla al origen de datos y prepara el servicio de P3.6.

Relación con la teoría: 3.3 desarrolla el contrato `[Parameter]`, la composición y la propiedad de los datos. Al terminar debes poder explicar por qué la tarjeta no necesita una ruta `@page` para funcionar dentro de la página.

Cuaderno de profundización P3.3 — Contrato del componente y pruebas de identidad

En el recorrido principal has sustituido el listado por tres instancias de TarjetaProducto. Esta ampliación opcional permite comprobar qué tan sólida es esa composición sin añadir API ni base de datos. Utiliza `PROYECTO/3.3/CursoWeb.Blazor` con el bloque de tarjetas activo y el `ul` original desactivado.

Caso A: tres tarjetas visibles, pero un único dato

Reproduce temporalmente la expresión equivocada `Producto="productos[0]"` en el padre. Verás tres instancias visuales que muestran la información de

Cuaderno. Es importante entender que se han creado **tres componentes**, pero se les ha transmitido **el mismo objeto**. Por eso contar tarjetas no basta como prueba de que la aplicación representa una colección correctamente.

Restablece `Producto="producto"` y comprueba que los atributos `data-producto-id` son diferentes. Un diagnóstico riguroso observa tanto la cantidad de elementos como la identidad y el contenido representado por cada uno.

Caso B: una tarjeta recibe un contrato incompleto

El hijo declara `[Parameter, EditorRequired] public Producto Producto`. Si el padre intentara representar `<TarjetaProducto />` sin proporcionar el parámetro, estaríamos rompiendo un contrato esperado del componente. No necesitas dejar esa versión en el proyecto; basta explicar que la tarjeta utiliza `Producto.Id` y `Producto.Nombre`, por lo que no puede producir una representación correcta sin recibir un producto.

`EditorRequired` ayuda a señalar el requisito desde las herramientas. No debes describirlo como una garantía absoluta de ejecución segura: el código sigue siendo responsable de gestionar entradas y estados válidos. La anotación comunica intención al autor del componente y al compilador o editor.

Caso C: el marcado cambia sin alterar la lógica del padre

En TarjetaProducto cambia temporalmente el texto «Referencia» por «Código». Compila y observa el resultado en las tres tarjetas. El padre no necesita saber cómo se etiqueta el identificador dentro de cada tarjeta. Después restaura el texto. Esa pequeña modificación muestra de forma concreta el beneficio de separar responsabilidades, incluso cuando todavía no hay métodos complejos ni servicios.

Ahora cambia temporalmente el valor del precio de Lápiz **en el padre**. Su tarjeta debe reflejarlo sin que cambies el código del hijo. El contrato se

conserva: TarjetaProducto recibe un objeto y lo representa. No sería adecuado copiar el precio a una constante privada dentro de la tarjeta, porque dejaría de reflejar el dato que recibe.

Caso D: el estilo falta, pero los datos son correctos

Abre el inspector del navegador. Si todos los elementos `article` existen con los Id correctos, pero carecen del aspecto de tarjeta, investiga `wwwroot/app.css` y la clase `tarjeta`. Esa incidencia se refiere a presentación visual, no al flujo de parámetros. Cambiar `Producto` por otro tipo no cargaría una hoja de estilos perdida.

También es posible lo contrario: las tarjetas pueden tener bordes y distribución correcta, pero mostrar datos equivocados. Entonces los estilos funcionan y la causa puede encontrarse en la expresión del padre o en el contenido de su lista.

Caso E: reorganizar elementos sin reescribir componentes

Imagina que el catálogo pasa a tener cuatro elementos. El `foreach` representará cuatro instancias del mismo componente. No es necesario copiar cuatro veces el bloque `<article>`. Esta es una ventaja funcional de la composición, además de su valor de mantenimiento. Si más adelante se reorganiza la lista, Blazor tiene mecanismos como `@key` para conservar identidad de componentes en escenarios donde importa; no añadimos esa dificultad al itinerario esencial, que trabaja con una lista pequeña y controlada.

Ejercicio complementario de marcado condicionado

El reto principal propone mostrar una marca cuando el precio supera cuatro euros. Como profundización, prueba tres valores para esa frontera: 3.99, 4.00 y 4.01. Con la condición `Precio > 4m`, solo 4.01 debe mostrar la marca. Con `Precio >= 4m`, también 4.00 la mostraría. Restaura los datos originales después de comprobarlo.

La diferencia importa porque el alumno no debe aceptar una prueba únicamente por mostrar el mensaje en Cuaderno. Es posible que una condición incorrecta produzca el mismo resultado para los tres datos iniciales. Diseñar valores alrededor de una frontera es una técnica de pruebas que ya conoces de C# y aplicas ahora al comportamiento de un componente.

Interpretación final

El padre decide **qué elementos** forman parte de la colección y el orden en que aparecen; el hijo decide **cómo se representa** cada elemento recibido. Esa separación no impide que la tarjeta tenga posteriormente un botón: solo significa que, cuando deba informar de una acción, necesitaremos un contrato de evento para no modificar silenciosamente el estado del padre. Ese será el problema central de P3.5.

P3.4. Eventos, @onclick, @bind y estado de componentes

Objetivo, ubicación y relación con P3.3

En P3.3 las tarjetas mostraban productos mediante parámetros. Ahora veremos que los componentes no solo representan información: también pueden reaccionar a eventos y alterar datos en memoria. Para aislar el concepto, durante P3.4 los botones de selección estarán escritos **directamente en la página padre**. En P3.5 los trasladaremos al hijo y aprenderemos EventCallback. No introducimos una API remota, guardado en EF Core ni administración de estado global.

La ubicación exacta es `M03_BLAZOR/PROYECTO/3.4/CursoWeb.Blazor/`, y el archivo principal `Components/Pages/Productos.razor`. El estado inicial contiene tres tarjetas, un texto «Seleccionado: ninguno», botones comentados y métodos C# asociados también comentados. **La compilación inicial debe funcionar sin activar ninguno de ellos**.

3.4.1. Abrir el estado inicial y reconocer el campo de selección

Detén la aplicación de P3.3. Abre el archivo `CursoWeb.Blazor.csproj` de la carpeta 3.4, ejecuta `dotnet build` y arranca con `dotnet run --no-launch-profile --urls http://localhost:5092`. Visita `http://localhost:5092/productos` y localiza el elemento `id="seleccion"`. Debes leer «Seleccionado: ninguno». Cuenta los tres artículos con clase `tarjeta`; no debe haber botones Seleccionar activos.

Abre el archivo de página y localiza la directiva `@rendermode InteractiveServer`. Encuentra también el campo:

```
private string seleccion = "ninguno";
```

El campo pertenece al componente y sirve para mostrar el valor actual. No es una propiedad de `Producto`, no se guarda en LocalDB y no se transmite a

un controlador. Esta observación anticipa el paso siguiente: un evento modificará ese campo y el framework volverá a representar el texto.

3.4.2. Activar el evento de selección

En `Components/Pages/Productos.razor`, localiza el comentario Razor identificado como `PASO 3.4.2 (ACTIVAR)`. El botón real del proyecto aparece en una única línea:

```
@* <button @onclick="() => Seleccionar(producto.Id)">Seleccionar</button> *@
```

Retira exclusivamente `@*` y `*@`, dejando el botón activo. En el bloque `@code`, localiza el fragmento delimitado por `PASO 3.4.2 INICIO` y `PASO 3.4.2 FIN` y activa su método:

```
private void Seleccionar(int id)
{
    seleccion = productos.First(p => p.Id == id).Nombre;
}
```

Compila. En `/productos` debe aparecer un botón Seleccionar por tarjeta. Pulsa en Lápiz y observa «Seleccionado: Lápiz». Después pulsa en Carpeta y observa el cambio a ese nombre. **El dato cambia dentro de la misma página** y no se necesita realizar una nueva operación MVC por cada clic.

Caso negativo de activación: si activaste el botón pero no el método C#, el compilador puede comunicar que `Seleccionar` no existe. Revisa ambos bloques del mismo paso; no inventes un método con otro nombre. **Caso negativo de ejecución:** si el botón está visible pero no produce cambios, comprueba primero que estás ejecutando `/productos` del proyecto 3.4, y después que el circuito Interactive Server funciona; recuerda la comparación de P3.1.

Por qué funciona: la expresión lambda captura el Id del producto que se está representando en esa tarjeta. El método del componente encuentra su nombre y modifica `seleccion`. Blazor programa una actualización de la

representación al finalizar el evento. No es necesario llamar manualmente a `StateHasChanged()` para este clic ordinario.

3.4.3. Incorporar un campo con @bind

Localiza `PASO 3.4.3 (ACTIVAR)` y retira los delimitadores de comentario del campo:

```
<label>Nombre nuevo:
    <input id="nombre-nuevo" @bind="nuevoNombre" />
</label>
```

En `@code` activa también el fragmento `PASO 3.4.3 INICIO / FIN`, que declara:

```
private string nuevoNombre = string.Empty;
```

Compila, abre `/productos` y escribe «Regla». De momento **no se añade** ninguna tarjeta porque todavía falta el método del paso siguiente. Esta separación resulta intencionada: estamos comprobando el vínculo entre el control HTML y el estado C#, sin introducir simultáneamente todas las operaciones.

A diferencia de `asp-for` del formulario MVC, aquí `@bind` participa en el enlace de datos de una interfaz interactiva. Si escribes sin abandonar el campo, recuerda que el enlace predeterminado del `input` se actualiza conforme a su evento de cambio; el itinerario principal no necesita la variante avanzada `@bind:event="oninput"`.

3.4.4. Activar el alta de un producto en memoria

Localiza `PASO 3.4.4 (ACTIVAR)` y descomenta exactamente el botón:

```
<button id="agregar" @onclick="Agregar">Añadir producto</button>
```

A continuación, en `@code`, activa **todo** el bloque `PASO 3.4.4 INICIO / FIN`, incluida la declaración del nuevo identificador:

```
private int siguienteId = 4;

private void Agregar()
{
    if (string.IsNullOrWhiteSpace(nuevoNombre))
        return;
    productos.Add(new Producto(siguienteId++, nuevoNombre.Trim(), 5m));
    nuevoNombre = string.Empty;
}
```

Ahora compila y ejecuta. Escribe «Regla» y pulsa Añadir. Debe aparecer una cuarta tarjeta y vaciarse el campo. Comprueba que el nuevo producto tiene un Id distinto de los tres anteriores y el precio fijo 5.00 que proporciona esta demostración.

Caso negativo: intenta añadir un nombre vacío y después uno compuesto exclusivamente por espacios. En ambos casos la cantidad de tarjetas debe mantenerse igual. No confundas esta condición sencilla con todas las reglas DataAnnotations de M02: aquí aprendemos el ciclo de eventos y la representación, no repetimos todo el CRUD persistente.

Estado persistente: recarga la ruta completamente para iniciar otra instancia de componente. Lo normal es que reaparezcan los tres productos originales. Es coherente: la lista pertenece a la sesión/instancia de interfaz y no se almacena en SQL ni se envía a la API que todavía no existe. Un resultado visual de «cuatro tarjetas» no demuestra un INSERT.

3.4.5. Editor de nombre: sustituir el valor de una tarjeta, no añadir otra

Objetivo. Completar el editor simple previsto para P3.4. Una vez que sabes seleccionar un producto, escribir en un input y añadir un elemento en memoria, aplicarás las mismas ideas a una operación diferente: **editar el nombre de un producto ya existente conservando su Id y su precio**. No se utiliza EF Core ni se realizan peticiones HTTP.

Punto de partida. Trabaja en `M03_BLAZOR/PROYECTO/3.4/CursoWeb.Blazor/Components/Pages/Productos.razor` con los pasos

3.4.2, 3.4.3 y 3.4.4 activados. Comprueba que seleccionar Lápiz cambia el texto y que añadir Regla crea una cuarta tarjeta. Para realizar la edición de manera controlada, recarga la página y comienza con los tres productos de ejemplo. No ejecutes otros procesos de la misma aplicación en el puerto 5092.

Paso A: comprender por qué el nombre no identifica correctamente una edición. En el estado anterior `Seleccionar(int id)` buscaba el producto y almacenaba su nombre en `seleccion`. Esa información permite mostrar una frase, pero no es una referencia suficientemente robusta para editar. Dos productos podrían tener el mismo nombre. Por ello, el nuevo manejador recordará el Id seleccionado en `idEnEdicion`, mientras un segundo campo guardará el texto que se propone escribir.

Paso B: activar el estado y los métodos del editor. Localiza en `@code` el bloque C# marcado `PASO 3.4.5 INICIO` y `PASO 3.4.5 FIN`. Elimina esas dos líneas delimitadoras, dejando activo el código interior. Contiene los campos:

```
private int? idEnEdicion;
private string nombreEditado = string.Empty;
```

Y un método que prepara el editor:

```
private void SeleccionarParaEditar(int id)
{
    var producto = productos.First(p => p.Id == id);
    idEnEdicion = id;
    seleccion = producto.Nombre;
    nombreEditado = producto.Nombre;
}
```

Fíjate en que el método no altera `productos`. Primero obtiene el objeto por identidad, después conserva el Id, y por último coloca el nombre tanto en la frase de selección como en el campo de edición. Este último valor será la base sobre la que el alumno escriba un cambio.

Paso C: sustituir el botón Seleccionar. Dentro del `foreach` de tarjetas existen ahora dos versiones de botón. La del paso 3.4.2 estará activa:

```
<button @onclick="() => Seleccionar(producto.Id)">Seleccionar</button>
```

Coméntala usando `@*` y `*@`. Activa en su lugar la variante del paso 3.4.5, preparada justo debajo:

```
<button @onclick="() => SeleccionarParaEditar(producto.Id)">
    Seleccionar
</button>
```

Importante: el segundo botón debe seguir estando **dentro del bucle y del artículo de cada producto**. No lo copies fuera del `foreach`, porque entonces la variable `producto` no tendría el mismo alcance. Mantén una sola versión activa por tarjeta para que la interfaz no muestre dos botones Seleccionar innecesarios.

Paso D: activar el campo y el botón de guardado. En la misma página, después de las tarjetas, localiza los comentarios del paso 3.4.5 correspondientes a:

```
<label>Editar nombre:
    <input id="editar-nombre" @bind="nombreEditado" />
</label>
<button id="guardar-cambio" @onclick="GuardarCambio">
    Guardar nombre
</button>
```

El archivo original tiene estos elementos escritos en una línea cada uno. Retira sus delimitadores de comentario Razor, conserva los identificadores y comprueba que el método `GuardarCambio` también está activo dentro del bloque C#. Si activas el botón sin el método, tendrás un error de compilación: la recuperación correcta es activar el código del mismo paso, no crear otro método de nombre distinto.

Paso E: comprender el código de GuardarCambio antes de pulsar. El método contiene un retorno anticipado si `idEnEdicion` es null o el texto está vacío. Después busca la posición del producto mediante el Id:

```
var indice = productos.FindIndex(p => p.Id == idEnEdicion.Value);
if (indice < 0)
    return;
```

Si la identidad no aparece en la lista, la operación no debe modificar arbitrariamente otro producto. A continuación, el código sustituye el valor del record en esa posición:

```
productos[indice] = productos[indice] with
{
    Nombre = nombreEditado.Trim()
};
seleccion = productos[indice].Nombre;
```

La expresión `with` conserva las otras propiedades del record original. No utilizamos `productos.Add`, porque añadir produciría un cuarto objeto y estaríamos realizando otra operación distinta de editar.

Paso F: compilar y ejecutar la edición positiva. Guarda todos los cambios y ejecuta `dotnet build`. Abre `http://localhost:5092/productos`. Comprueba que existen tres tarjetas y que el nuevo campo de edición está disponible. Pulsa Seleccionar en Lápiz. El campo `editar-nombre` debe mostrar «Lápiz». Escribe «Lápiz rojo» y pulsa Guardar nombre.

Resultados esperados: la tarjeta con `data-producto-id="2"` debe mostrar «Lápiz rojo». El precio debe seguir siendo 1.20 y el total de tarjetas debe continuar siendo tres. La línea de selección también debe actualizarse a «Seleccionado: Lápiz rojo». La operación no crea un nuevo producto: sustituye el valor del que ya tenía ese Id. Comprueba todos esos resultados y no te conformes únicamente con que el botón parezca responder.

Paso G: ejecutar un caso negativo. Con el mismo producto seleccionado, borra el campo de edición, escribe tres espacios y pulsa Guardar nombre. El código contiene `string.IsNullOrWhiteSpace`, por lo que el producto debe conservar «Lápiz rojo». Otra posibilidad negativa es intentar guardar antes de seleccionar ningún producto en una nueva carga de la página: `idEnEdicion` será null y el método regresará sin hacer modificaciones.

Paso H: contrastar edición y alta. En el mismo componente siguen existiendo `nuevoNombre` y el botón Añadir. La alta incrementa el tamaño de la colección y asigna un Id nuevo. La edición conserva el tamaño e identidad, pero modifica una propiedad del objeto existente. Escribe en tu cuaderno dos secuencias: entrada → Añadir → cuarta tarjeta y selección → entrada de edición → Guardar → mismo Id con nombre distinto. Las dos usan C# y Razor, pero aplican operaciones de estado diferentes.

Diagnóstico 1 — se crean cuatro tarjetas. Revisa si, por error, utilizaste `productos.Add` dentro de `GuardarCambio`. El proyecto canónico utiliza `productos[indice] = ...`. La sustitución no debería incrementar la cantidad de elementos. Si quieres verificar la diferencia, inspecciona `productos.Count` antes y después del evento.

Diagnóstico 2 — cambia el producto equivocado. Comprueba que has activado `SeleccionarParaEditar(producto.Id)` dentro del bucle y que `FindIndex` compara el Id recibido. Si buscaras por `Nombre`, un catálogo con nombres duplicados haría ambigua la elección. El identificador existe precisamente para evitar esa situación.

Diagnóstico 3 — la edición no se refleja hasta recargar. Comprueba que la página declara `@rendermode InteractiveServer` y que el botón ejecuta `GuardarCambio`. Verifica que el campo de edición actualiza `nombreEditado` y que se asigna el record modificado a la lista que representa el `foreach`. No deberías necesitar una recarga completa para ver el resultado de un evento local correctamente conectado.

Diagnóstico 4 — la selección sigue mostrando el nombre antiguo. La edición podría estar sustituyendo el objeto sin actualizar la frase de selección. Revisa la asignación final `seleccion = productos[indice].Nombre`. Esta es una regla de consistencia de presentación que pertenece al componente dueño del estado, no a una función de CSS ni a la infraestructura HTTP.

Diagnóstico 5 — un nombre compuesto por espacios borra el original. Comprueba la condición de entrada `string.IsNullOrWhiteSpace(nombreEditado)` antes de modificar la colección. La versión preparada no guarda valores vacíos. No hace falta construir un formulario MVC con DataAnnotations para practicar esta condición básica dentro del editor.

Conclusión pedagógica: P3.4.5 combina las ideas anteriores para mostrar un caso distinto de interacción real. Puedes conservar el Id, modificar un nombre, rechazar un cambio inválido y observar una representación coherente sin incorporar ninguna tecnología de M04. La teoría 3.4, apartado «Un editor real en memoria», explica por qué este resultado es diferente de un CRUD persistente.

3.4.6. Reto integrador — Catálogo consultable con filtros combinados y ordenación

Situación de trabajo. La tienda que utiliza este catálogo quiere poder localizar un producto sin perder de vista qué artículos tiene realmente almacenados en la sesión. El encargado necesita buscar por parte del nombre, limitar el precio y cambiar el orden. No quiere que los filtros eliminen productos ni que la edición del paso anterior deje de funcionar. Resolverás este encargo **en el mismo componente Productos.razor de P3.4**, sin añadir una API, una base de datos ni un segundo listado independiente.

Resultado que debes conseguir: al abrir la página aparecen tres tarjetas, ordenadas inicialmente de menor a mayor precio. Puedes invertir ese orden, escribir una parte del nombre para filtrar mientras tecleas y

aplicar un precio máximo. El resumen visible indica cuántos elementos se muestran respecto al total existente. Tras borrar los criterios, los tres productos reaparecen. El botón de edición de 3.4.5 sigue conservando los Id.

Preparación. Completa y comprueba 3.4.2–3.4.5 antes de este ejercicio. En el archivo M03_BLAZOR/PROYECTO/3.4/CursoWeb.Blazor/Components/Pages/Productos.razor hay un bloque C# comentado con las marcas RETO INTEGRADOR 3.4.6 INICIO y FIN, cuatro elementos Razor comentados cerca del principio y una variante de la expresión foreach. No inventes nombres diferentes: las piezas están preparadas para activarse conjuntamente.

Paso 1 — Anticipar el resultado sobre datos concretos. Sin modificar nada, escribe qué orden esperas obtener al clasificar por precio ascendente los tres productos originales: Cuaderno vale 4,50 €, Lápiz 1,20 € y Carpeta 3,75 €. Después escribe el orden descendente. Esta predicción es importante porque, si más adelante la pantalla muestra otro orden, podrás distinguir un error en la consulta de un error de representación. El orden ascendente esperado es Lápiz, Carpeta y Cuaderno; el descendente es Cuaderno, Carpeta y Lápiz.

Paso 2 — Activar la proyección de consulta. En el bloque RETO INTEGRADOR 3.4.6 retira los dos delimitadores del comentario C#. El código contiene tres variables y una propiedad calculada:

```
private string busqueda = string.Empty;
private decimal? precioMaximo;
private bool ordenAscendente = true;
```

La propiedad ProductosVisibles ordena por Precio, usa Id como segundo criterio estable y después filtra con Where. En el predicado, la condición de nombre permite que una búsqueda vacía no restrinja la lista; la condición numérica solo se aplica cuando precioMaximo tiene un valor. Lee el

predicado completo antes de compilar. ¿Por qué un filtro vacío no debe dejar la pantalla sin productos?

Paso 3 — Mostrar los controles de la consulta. Descomenta **cada una** de las cuatro líneas Razor con el comentario RETO INTEGRADOR 3.4.6: el campo id="buscar" enlazado a busqueda con evento oninput; el campo id="precio-maximo" enlazado a precioMaximo; el botón id="alternar-orden" que invoca AlternarOrden; y el párrafo id="resumen-filtrado" con el número visible y el número total. No copies los controles por segunda vez; activa los que ya están escritos.

Paso 4 — Conectar consulta y representación. Localiza la instrucción activa del bucle de tarjetas:

```
@foreach (var producto in productos)
```

Cambia **solo esa expresión** por:

```
@foreach (var producto in ProductosVisibles)
```

Conserva intactos el artículo, su atributo data-producto-id, el precio y el botón Seleccionar. El archivo incluye una variante comentada a modo de pista: no debes dejar dos bucles activos. La colección productos sigue existiendo y se utiliza como fuente de datos; lo único que cambia es qué secuencia se recorre al dibujar las tarjetas.

Paso 5 — Comprobar el encargo en el navegador. Guarda y compila el proyecto de 3.4. Visita la ruta /productos. Deberías leer «Mostrando 3 de 3 productos» y observar primero Lápiz. Pulsa «Invertir orden por precio»: ahora la primera tarjeta debe ser Cuaderno, pero los precios y los Id siguen iguales. Escribe «láp» en la búsqueda; el contador debe pasar a 1 de 3 y solo queda Lápiz, aunque no hayas cambiado la colección. Escribe 1 como precio máximo y abandona el campo: no debe aparecer ninguna tarjeta, porque 1,20 es mayor que 1. Sustituye el máximo por 2 y abandona el

campo: Lápiz reaparece. Borra la búsqueda: sigue siendo visible solo Lápiz, esta vez por el criterio de precio. Borra también el límite: reaparecen los tres artículos.

Paso 6 — Relacionar filtros y editor. Selecciona Lápiz, comprueba que el editor del paso 3.4.5 recibe su nombre, escribe «Lápiz rojo» y guarda. Ahora utiliza la búsqueda «rojo»: el producto editado debe aparecer sin necesidad de volver a crear la lista. Anota cuál es su Id antes y después de editar. Si cambió, has construido un alta en lugar de una edición. Si no se encuentra con el nuevo nombre, revisa si ProductosVisibles lee la colección original o alguna copia que hayas mantenido por accidente.

Comportamientos incorrectos que debes saber diagnosticar. Si al borrar la búsqueda siguen faltando productos, comprueba que no ejecutaste RemoveAll o una asignación destructiva para filtrar. Si el nombre se filtra solo al salir del input, revisa la presencia de @bind:event="oninput". Si el precio parece no cambiar hasta que sales del control, recuerda que en este campo hemos utilizado el evento por defecto. Si los filtros muestran correctamente el contador pero no las tarjetas, comprueba qué expresión recorre realmente el foreach. Si seleccionar o guardar deja de funcionar tras ordenar, confirma que el botón pasa producto.Id y no la posición de la tarjeta.

Cierre razonado. Explica en un párrafo por qué ordenar y filtrar no deberían modificar el contenido de la colección; contrástalo con la edición de nombre, que sí sustituye un elemento. Distingue el total de objetos en memoria del subconjunto visible y describe qué se reinicia al recargar completamente la página. Este reto prepara la separación entre datos y criterios de presentación que reutilizaremos cuando el origen se convierta en una API.

Reto 3.4 — Limpiar la selección sin alterar el catálogo

El proyecto contiene un botón y un método comentados bajo la marca `RETO 3.4`. Su objetivo es practicar que un mismo estado puede cambiar por otra acción distinta de Seleccionar.

Activa en el Razor:

```
<button id="limpiar" @onclick="Limpiar">Limpiar selección</button>
```

Y activa en `@code`:

```
private void Limpiar() => seleccion = "ninguno";
```

Compila. Selecciona Cuaderno, comprueba que la pantalla muestra su nombre y pulsa Limpiar. El texto debe volver a «Seleccionado: ninguno». **Control negativo:** ninguna tarjeta debe desaparecer ni cambiar de precio. El método solo asigna un nuevo valor a `seleccion`; no tiene motivos para modificar la lista.

Solución razonada: el botón y el método están dentro del componente dueño de ese estado, por lo que no necesitamos EventCallback. Si se activara la línea del botón sin el método, el error de compilación indicaría una correspondencia incompleta entre vista y código. Si se cambiara `seleccion` pero no la interfaz, revisaríamos el contexto interactivo antes de introducir una operación de red.

Profundización aplicada: diagnóstico de cinco síntomas parecidos

Una interfaz que «no hace nada» puede fallar por causas distintas. Si el botón no aparece, inspecciona el comentario Razor. Si aparece pero no compila, comprueba el método correspondiente. Si compila pero el clic no actualiza, confirma el modo interactivo y la conexión. Si el clic llega a `Agregar` pero no crea una tarjeta, verifica el valor de `nuevoNombre` y la condición `IsNullOrWhiteSpace`. Si el objeto se añade y no se representa, comprueba que el `foreach` lee la misma colección a la que se ha agregado.

La lección es localizar la fase exacta: **entrada → enlace → evento → estado → representación**. No modifiques `Program.cs` cada vez que un dato es incorrecto, ni cambies el modelo cuando lo que falta es activar un botón. La teoría 3.4 explica las responsabilidades de cada pieza y el comportamiento esperado cuando no existe persistencia.

Laboratorio de profundización P3.4 — Distinguir entrada, evento y representación

El recorrido principal ya ha permitido seleccionar productos, escribir un nombre y añadirlo a la lista en memoria. Esta actividad opcional está pensada para quien necesita entender por qué cada operación funciona y qué hacer cuando una parte falla. Se ejecuta sobre la carpeta `PROYECTO/3.4/CursoWeb.Blazor`, después de activar los pasos 3.4.2–3.4.4 y, si lo deseas, el reto de limpiar selección. No crees un servicio ni otra página: todos los fenómenos pueden observarse en el componente existente.

Ensayo A: un evento modifica una cadena, no la colección

Con los tres productos iniciales representados, selecciona Cuaderno. Comprueba que el texto cambia a «Seleccionado: Cuaderno» y cuenta las tarjetas. Deben continuar siendo tres. Selecciona después Lápiz y comprueba que el número de tarjetas tampoco cambia. Con este experimento podemos demostrar que la operación `Seleccionar(int id)` solo lee la colección y asigna un nuevo valor a `seleccion`.

Abre `Components/Pages/Productos.razor` y localiza el método. ¿Hay algún `productos.Add` o `productos.RemoveAll` dentro de `Seleccionar`? No. Por tanto, un cambio en el contador de tarjetas indicaría otra modificación adicional o una versión de código distinta de la prevista. El diagnóstico correcto consiste en verificar el método y la colección representada, no en asumir que Blazor modifica automáticamente todos los campos cuando cambia uno.

Resultado razonado: el mismo componente puede mantener varios datos independientes. Un evento debe producir los efectos que define su

método; la nueva representación solo refleja el estado resultante. Esta explicación servirá para comprender por qué, en P3.5, eliminar una tarjeta también necesita decidir qué hacer con la selección actual.

Ensayo B: el campo visual y el campo de C# están enlazados

Localiza el input con `id="nombre-nuevo"` y escribe «Marcador». Antes de pulsar Añadir, abandona el campo para confirmar el evento de cambio del enlace. Con un punto de interrupción en el código puedes inspeccionar el valor de `nuevoNombre` cuando se ejecuta `Agregar`. Debería contener el texto introducido.

No confundas el contenido del DOM con el miembro C# si la aplicación no está conectada interactivamente. El atributo `@bind` participa en mantener esa relación, pero también necesita el modo interactivo para coordinar los cambios del componente en este escenario. Si el botón Añadir recibe un texto anterior, investiga cuándo se dispara el evento de enlace y si la entrada cambió realmente antes del clic.

Como ampliación conceptual, consulta la documentación de `@bind:event="oninput"`. No es obligatorio modificar el proyecto, pero debes saber que el momento de actualización puede ser configurable y que eso afecta a lo que observa el método en ciertos patrones de interacción.

Ensayo C: un nombre inválido no genera una tarjeta

Con el campo vacío, pulsa Añadir. Comprueba que no aparece una tarjeta nueva. Escribe solo espacios y repite. Después escribe «Regla» y verifica que sí se añade. Los tres resultados delimitan la condición `string.IsNullOrWhiteSpace`: no basta con que exista una cadena; debe contener texto significativo después de la comprobación utilizada en este laboratorio.

La prueba no pretende acreditar la misma validación que M02: allí estudiamos DataAnnotations y ModelState para formularios. Aquí estamos aprendiendo cómo un manejador de componente usa un dato enlazado y

modifica una colección. El criterio de éxito es observar el efecto real de `Agregar` o su ausencia, no recibir un código HTTP específico.

Solución razonada: una entrada vacía produce un retorno anticipado del método. Una entrada válida alcanza `productos.Add` y después limpia `nuevoNombre`. El control del caso negativo forma parte de la misma función que modifica el estado, de modo que no debemos escribir una excepción de interfaz nueva para este ejercicio.

Ensayo D: el contador local y su identidad

Añade dos productos válidos de manera consecutiva, por ejemplo «Regla» y «Marcador». Comprueba que ambos aparecen y que cada tarjeta recibe un `data-producto-id` distinto. El campo `siguienteId` comienza en 4 y se incrementa después de cada alta. Esto permite distinguir visualmente los objetos durante la vida del componente.

No extrapoles ese contador al ámbito de una base de datos utilizada por miles de usuarios. Aquí no existe concurrencia entre múltiples procesos que comparten un identificador global ni un mecanismo de persistencia. La decisión técnica es adecuada para una lista controlada de laboratorio, y el alumno debe saber explicar esa limitación.

Si los dos objetos aparecen con el mismo Id, comprueba dónde se declara y actualiza `siguienteId`. No debería reinicializarse dentro de `Agregar` en cada clic. Si el nombre aparece repetido, revisa que el campo `nuevoNombre` se limpia después de añadir. La clase de problema determina qué línea merece investigarse.

Ensayo E: el estado visible deja de existir al recargar

Selecciona Lápiz, añade Regla y observa los cambios. Ahora ejecuta una recarga completa de la dirección. Puede reaparecer el catálogo inicial con tres elementos y la selección «ninguno». El resultado se explica por la

creación de otra instancia de componente y por la ausencia de almacenamiento externo. No hay ninguna operación de `SaveChangesAsync`.

Compara explícitamente con P2.4: en aquel módulo se verificaba la persistencia después de reiniciar Kestrel y consultar LocalDB; aquí la prueba positiva es el cambio visible en la instancia activa. **Una aplicación puede cumplir correctamente su objetivo sin que todos sus campos sean persistentes**. El problema aparece cuando se confunde el alcance del estado con una promesa de almacenamiento que el diseño nunca realizó.

Ensayo F: dos botones que no deben tener el mismo efecto

Activa el reto de Limpiar. Selecciona Cuaderno y pulsa Limpiar. La selección vuelve a «ninguno» sin reducir el catálogo. Selecciona Lápiz y pulsa Añadir con un nombre válido: la cantidad aumenta, pero la selección continúa en Lápiz salvo que el propio código la cambie. Esta diferencia demuestra que `Limpiar` y `Agregar` operan sobre campos distintos.

Una solución que limpiara automáticamente la selección en todos los eventos podría ser aceptable como otra decisión de diseño, pero no es lo que la práctica especifica. Para verificar reproducibilidad hay que contrastar el comportamiento con el código que realmente se entrega, no con suposiciones sobre lo que debería hacer cualquier interfaz.

Resolver un reporte de error de otro alumno

Supongamos que alguien informa: «Puedo ver las tarjetas, pero el botón Añadir no funciona». No respondas directamente «reinstala Blazor». Pregunta qué ocurre exactamente: ¿aparece el botón? ¿compila el método? ¿el campo conserva el texto? ¿el clic produce algún cambio? ¿la colección aumenta en el depurador? ¿la página tiene circuito interactivo? Cada respuesta acota una fase distinta.

Si el botón ni siquiera aparece, revisa el comentario Razor. Si existe pero no compila, busca el método `Agregar`. Si compila pero no se ejecuta, revisa modo y conexión. Si ejecuta y retorna, revisa el valor de `nuevoNombre`. Si añade el objeto pero la pantalla no lo representa, revisa el bucle y la actualización del estado. La solución no necesita introducir tecnologías nuevas: requiere comprender el camino completo.

Producto del ejercicio de transferencia

Redacta una explicación de un máximo de una página que siga «escribir Regla → pulsar Añadir → aparecer cuarta tarjeta». Debe nombrar `@bind`, `nuevoNombre`, `@onclick`, `Agregar`, `productos` y el `foreach`, asignando a cada uno una responsabilidad. Contrasta con el recorrido «seleccionar Lápiz → cambiar texto». Si puedes señalar que el segundo no modifica la lista y el primero sí, has comprendido el modelo de estado local de P3.4.

P3.5. Comunicación padre–hijo mediante EventCallback

Objetivo y preparación

En P3.4 el botón estaba en el mismo componente que poseía la lista. Ahora queremos que cada `TarjetaProducto` muestre su propio botón, pero que sea `Productos.razor` quien actualice la selección o quite el elemento. Esta diferencia nos obliga a expresar un **contrato de evento del hijo hacia el padre**. El proyecto se encuentra en `M03_BLAZOR/PROYECTO/3.5/CursoWeb.Blazor/`.

El estado inicial ya compila con tres tarjetas, «Seleccionado: ninguno» y «Total: 3». Los dos archivos que cambiarán son `Components/TarjetaProducto.razor` y `Components/Pages/Productos.razor`. Evita introducir una colección global o compartir campos a través de un singleton; para esta relación directa basta `EventCallback<int>`.

3.5.1. Inspeccionar la propiedad de los datos

Detén P3.4, abre 3.5, compila y ejecuta. Comprueba el estado inicial. Abre `Productos.razor` y señala dónde se crea la lista. Abre `TarjetaProducto.razor` y localiza:

```
[Parameter, EditorRequired] public Producto Producto { get; set; } = default!;
```

Esta propiedad recibe un dato desde el padre. Ahora queremos otro contrato: el hijo comunicará un Id, pero **no será dueño de la lista que el padre representa**. Antes de activar código, dibuja el flujo: parámetro Producto desde padre a hijo; evento de selección desde hijo a padre.

3.5.2. Declarar el callback en la tarjeta

En `Components/TarjetaProducto.razor` activa el bloque C# delimitado por `PASO 3.5.2 INICIO / FIN`:

```
[Parameter] public EventCallback<int> AlSeleccionar { get; set; }
```

```
private Task NotificarSeleccion() =>
    AlSeleccionar.InvokeAsync(Producto.Id);
```

Activa también el botón Razor señalado por `PASO 3.5.2 (ACTIVAR)`:

```
<button @onclick="NotificarSeleccion">Seleccionar</button>
```

El botón invoca un método del propio hijo; ese método notifica al callback mediante `InvokeAsync`. De momento no has cambiado el padre. La unidad funcional completa requiere conectar el callback al método que modificará la selección.

3.5.3. Conectar el callback al padre

En `Components/Pages/Productos.razor` activa el bloque C# `PASO 3.5.2 INICIO / FIN`:

```
private void Seleccionar(int id)
{
    seleccion = productos.First(p => p.Id == id).Nombre;
}
```

Busca la línea que representa cada tarjeta:

```
<TarjetaProducto Producto="producto" />
```

El proyecto contiene debajo una variante comentada. **Sustituye la línea activa** por:

```
<TarjetaProducto Producto="producto" AlSeleccionar="Seleccionar" />
```

No debes dejar activas ambas versiones, porque obtendrías seis tarjetas. Guarda, compila y abre `/productos`. Pulsa Seleccionar en Lápiz. Debe aparecer «Seleccionado: Lápiz» en el componente padre. Pulsa Carpeta y comprueba que cambia el texto.

Explicación: el hijo no modifica directamente `seleccion`. Comunica el identificador con `EventCallback<int>`; el padre recibe el dato, busca el nombre y

cambia un campo propio. Esa separación permite reutilizar TarjetaProducto sin acoplarlo a una única pantalla.

3.5.4. Reconstruir la comunicación desde el navegador

Sigue un clic con puntos de interrupción en Visual Studio si lo deseas. El orden debe ser: botón de la tarjeta → `NotificarSeleccion` → `AlSeleccionar.InvokeAsync(Producto.Id)` → método `Seleccionar(int)` del padre → campo `seleccion` → texto actualizado. No hay un nuevo controlador ni una nueva acción POST como en M02.

Como caso negativo de código, si la propiedad `AlSeleccionar` está comentada en la tarjeta pero el padre intenta pasar ese parámetro, puedes obtener un error de compilación o de parámetro desconocido. Si el botón no aparece, revisa el comentario Razor. Si aparecen duplicados, revisa las dos variantes de TarjetaProducto en el padre.

Reto 3.5 — Solicitar que se quite una tarjeta

Enunciado: cada tarjeta tendrá un botón «Quitar». Al pulsarlo, la tarjeta **solicita** al padre que elimine ese Id de la lista en memoria. No debe modificar una colección privada del padre por acceso oculto.

En `Components/TarjetaProducto.razor` activa el bloque `RETO 3.5 INICIO / FIN`:

```
[Parameter] public EventCallback<int> AlQuitar { get; set; }

private Task NotificarQuitar() =>
    AlQuitar.InvokeAsync(Producto.Id);
```

Y activa el botón del reto:

```
<button @onclick="NotificarQuitar">Quitar</button>
```

En `Components/Pages/Productos.razor` activa el bloque `RETO 3.5 INICIO / FIN`:

```
private void Quitar(int id)
{
    productos.RemoveAll(p => p.Id == id);
```

```
    seleccion = "ninguno";
}
```

Por último, **reemplaza** la versión activa de TarjetaProducto por la tercera variante preparada en el proyecto:

```
<TarjetaProducto Producto="producto"
                 AlSeleccionar="Seleccionar"
                 AlQuitar="Quitar" />
```

Compila y ejecuta. Selecciona Lápiz y comprueba que el padre refleja su nombre. Ahora pulsa Quitar en esa tarjeta: el total debe pasar de tres a dos y la tarjeta de Lápiz debe desaparecer. Además, la selección vuelve a «ninguno».

Caso negativo: al recargar completamente el componente pueden volver los tres productos, porque la lista es local. No interpretes esta operación como un DELETE de SQL Server. La persistencia no es el objetivo de P3.5.

Solución razonada: la tarjeta comunica únicamente una intención y el Id del producto. El padre es dueño de la lista, por lo que decide cómo aplicar `RemoveAll`. Esta solución es coherente con el flujo de datos padre→hijo mediante `[Parameter]` y la notificación hijo→padre mediante `EventCallback<int>`. La teoría 3.5 contiene la explicación de ambos contratos.

3.5.5. Reto integrador — Quitar y deshacer sin perder la posición original

Encargo. El usuario puede retirar una tarjeta del catálogo por error. Añade una acción que permita **deshacer la última eliminación** y recuperar exactamente el objeto retirado, en la misma posición en la que aparecía. La tarjeta seguirá emitiendo un EventCallback con el Id y será el padre quien decida qué guardar, borrar o recuperar. El ejercicio no consiste en añadir otra colección al hijo ni en implementar un historial ilimitado de operaciones.

Preparación. Termina el paso 3.5.2 y el reto de eliminación de 3.5. Deben funcionar el botón Seleccionar y el botón Quitar de cada tarjeta; el

elemento padre debe mostrar tres productos y un contador. Comprueba que solo hay una etiqueta TarjetaProducto activa en cada iteración del foreach, con los parámetros Producto, AlSeleccionar y AlQuitar.

Paso 1 — Definir qué información se pierde al quitar. Comienza con Cuaderno, Lápiz y Carpeta. Si pulsas Quitar en Lápiz, no basta con recordar el nombre «Lápiz»: queremos recuperar su Id 2, su precio de 1,20 € y su posición intermedia. Escribe por qué no resolvería el problema limitarse a aumentar el contador o ejecutar productos.Add. Este último método insertaría al final, por lo que no restituiría necesariamente el orden que había antes de quitar.

Paso 2 — Activar el bloque del padre. En Components/Pages/Productos.razor localiza las marcas RETO INTEGRADOR 3.5.5 INICIO y FIN y retira únicamente las líneas delimitadoras del comentario C#. El bloque introduce:

```
private (Producto Producto, int Indice)? eliminacionPendiente;
```

También incluye TextoDeshacer, QuitarConDeshacer(int id) y DeshacerEliminacion(). El primer método busca el índice mediante FindIndex, guarda en eliminacionPendiente la pareja producto/posición, retira el objeto de la lista y limpia el texto de selección. El segundo comprueba que existe una eliminación pendiente, reintroduce el producto en la posición guardada y anula el registro para que no se pueda deshacer dos veces. No cambies la declaración de EventCallback del hijo.

Paso 3 — Redirigir una intención, no alterar el contrato del hijo. Dentro del foreach encontrarás la variante de la tarjeta que activaste en el reto anterior:

```
<TarjetaProducto Producto="producto"
                 AlSeleccionar="Seleccionar"
                 AlQuitar="Quitar" />
```

Sustituye exclusivamente el valor de AlQuitar por QuitarConDeshacer:

```
<TarjetaProducto Producto="producto"
                 AlSeleccionar="Seleccionar"
                 AlQuitar="QuitarConDeshacer" />
```

Conserva el parámetro AlSeleccionar y mantén una sola tarjeta por vuelta del bucle. Observa que Components/TarjetaProducto.razor no necesita saber si el padre está guardando un historial, un único registro o ningún registro: su función continúa siendo notificar el Id.

Paso 4 — Activar la interfaz de recuperación. En la página padre descomenta los dos elementos Razor del reto: el botón id="deshacer", que llama a DeshacerEliminacion y está deshabilitado cuando eliminacionPendiente es null, y el párrafo id="estado-deshacer" que muestra TextoDeshacer. El texto permite saber **qué producto se podría recuperar**, no únicamente que existe alguna operación pendiente. Si el compilador no encuentra uno de esos miembros, comprueba que el bloque C# también esté activo.

Paso 5 — Recorrer un caso completo con un estado esperado. Al entrar, el botón «Deshacer última eliminación» debe estar deshabilitado y el texto debe indicar que no hay eliminación pendiente. Pulsa Quitar en Lápiz. Ahora el contador debe ser 2, las tarjetas visibles serán Cuaderno y Carpeta, el botón de recuperación quedará habilitado y el texto pendiente identificará Lápiz. Púlsalo una vez. Deben reaparecer las tres tarjetas en este orden: Cuaderno, Lápiz, Carpeta. Verifica que Lápiz conserva el Id 2 seleccionándolo: el padre debe mostrar «Seleccionado: Lápiz». El botón Deshacer vuelve a quedar deshabilitado.

Paso 6 — Explorar los límites de esta solución. Quita primero Cuaderno y después Carpeta antes de utilizar Deshacer. Predice qué ocurrirá: solo se conserva la eliminación más reciente. Después del deshacer quedarán recuperados los datos de Carpeta, pero no los de

Cuaderno. Esto es deliberado y demuestra el alcance de **una única eliminación pendiente**, no de una pila de deshacer. Para recuperar el ejercicio inicial, vuelve a cargar /productos y empieza con tres productos. No prometas al usuario del caso un historial ilimitado que no has programado.

Diagnóstico de tres fallos plausibles. Si tras deshacer Lápiz aparece al final, comprueba que no sustituiste Insert por Add. Si aparece duplicado, revisa la comprobación de Id antes de insertar y que no dejaste dos handlers conectados. Si el botón de recuperación nunca se habilita, revisa el método asociado a AlQuitar: tal vez sigues llamando al antiguo Quitar, que borra sin rellenar eliminacionPendiente. Aclara cuál es la diferencia entre desactivar el botón por estado y proteger el método con un retorno temprano.

Pregunta de diseño. ¿Por qué mantenemos la tupla en Productos.razor y no en TarjetaProducto.razor? Responde describiendo qué componente es propietario de la colección, quién conoce la posición de cada artículo y quién puede reconstruir el orden. Esta explicación es tan importante como la ejecución del caso: demuestra que utilizas los parámetros y EventCallback para respetar una responsabilidad, no solo para pasar una función entre archivos.

Diagnóstico y consolidación

Si al activar el reto el proyecto falla, revisa que el parámetro `AlQuitar` exista en el hijo y que el método `Quitar` exista en el padre. Si se elimina otro elemento, revisa qué Id envía la tarjeta y qué predicado emplea `RemoveAll`. Si un botón de selección no cambia el texto, sigue la cadena `NotificarSeleccion` → `InvokeAsync` → `Seleccionar`. Si se dibujan seis tarjetas, desactiva las variantes de componente que dejaste duplicadas.

El aprendizaje principal no es memorizar una sintaxis: es mantener una correspondencia clara entre **quién posee el dato**, **quién recibe un**

parámetro y **quién comunica una intención**. Esta misma separación será útil cuando M04 conecte la interfaz a una API remota.

Laboratorio de profundización P3.5 — Dos contratos y un dueño del estado

Esta actividad opcional aprovecha el proyecto 3.5 con los bloques de selección y reto ya activados. No introduce ningún servicio ni endpoint adicional. Su finalidad es comprobar que la comunicación entre componentes es realmente explícita, tipada y coherente con la propiedad del estado.

Primer recorrido: entrada del padre hacia la tarjeta

En `Productos.razor` encuentra el `foreach` y la etiqueta `TarjetaProducto` que recibe `Producto="producto"`. Abre el hijo y localiza su propiedad pública marcada con `[Parameter]`. Escribe una explicación causal: cada iteración del padre suministra un objeto concreto y el hijo muestra nombre, precio e Id de ese objeto.

La tarjeta no posee la colección completa. Si decide mostrar únicamente los campos del producto recibido, puede reutilizarse donde haya otra colección semejante. Por ello conviene distinguir el **dato de entrada** del contrato de evento que utilizaremos después.

Caso negativo reversible: sustituir `Producto="producto"` por `Producto="productos[0]"` produciría varias tarjetas que muestran el primer producto. No es un problema de EventCallback ni de navegación, sino de información equivocada transmitida en el punto de composición. Restablece la expresión correcta si realizas el experimento.

Segundo recorrido: salida del hijo hacia el padre

Con los callbacks activos, pulsa Seleccionar en Lápiz. La tarjeta conoce `Producto.Id` y ejecuta `NotificarSeleccion`. Ese método invoca `AlSeleccionar.InvokeAsync(Producto.Id)`. El padre enlaza la propiedad `AlSeleccionar` con

su método `Seleccionar(int)` y actualiza el campo `seleccion`. La tarjeta no necesita cambiar el texto del padre por acceso directo; comunica un Id para que el padre decida.

El elemento visible `id="seleccion"` está fuera de la tarjeta, lo que hace observable la separación: el evento se origina dentro del hijo pero su efecto pertenece al estado del padre. Un sistema basado en referencias ocultas al componente contenedor sería más difícil de comprender y reutilizar.

Tercer recorrido: una solicitud de eliminación

Pulsa Quitar en la tarjeta de Lápiz. El hijo invoca `AlQuitar` con el mismo Id, y el padre elimina de su lista los objetos que coincidan. Debes observar tres efectos: la tarjeta desaparece, el total disminuye y la selección vuelve a «ninguno». Los tres dependen de que el padre actualice correctamente sus campos.

Si la tarjeta desaparece pero el texto sigue diciendo que Lápiz está seleccionado, `EventCallback` probablemente ha funcionado. El defecto se encontraría en el método `Quitar`, porque no ha limpiado `seleccion` como exige la solución del reto. Esta es una enseñanza importante: **comunicación correcta** y **regla de estado coherente** no son sinónimos.

Investigar por qué las tarjetas aparecen duplicadas

El proyecto contiene tres variantes de componente padre para permitir la evolución docente: sin callbacks, con selección y con selección y eliminación. Si dejas activas varias, cada producto puede representarse dos o tres veces. El síntoma es un número incorrecto de elementos HTML, pero la colección C# puede continuar teniendo exactamente tres objetos. No intentes corregirlo eliminando elementos de `productos`: revisa las etiquetas `TarjetaProducto` activas dentro del bucle.

La explicación causal se basa en composición, no en la lógica del callback. Cada etiqueta instancia un componente; dos etiquetas representan dos instancias. El sistema no sabe que una de ellas es «la versión antigua del ejercicio» si el alumno no la ha comentado.

Estudiar un tipo de callback incorrecto

El contrato `EventCallback<int>` expresa que el hijo notificará un entero. ¿Qué ocurriría si intentáramos pasar una cadena o un objeto no compatible con ese contrato? El compilador o el contrato de invocación ya no sería coherente. No es necesario alterar la práctica hasta dejarla rota; basta identificar en ambos archivos dónde se declara el tipo y qué valor se envía.

Esa disciplina de tipos aporta una ventaja importante: el padre puede diseñar su método receptor sabiendo qué clase de dato recibirá. Sin embargo, no hay que atribuirle validación de identidad, autorización o persistencia; un entero bien tipado podría no pertenecer a un producto de la lista, y una operación autorizada exigiría requisitos que todavía no estudiamos en M03.

Una propuesta alternativa que evitaremos

Imagina que la tarjeta almacenara una copia completa de todos los productos y eliminara por sí misma uno de ellos. Habría dos fuentes de estado, la del padre y la del hijo, que podrían evolucionar de forma distinta. Nuestro diseño evita ese problema porque la tarjeta solo conoce el objeto que presenta y los callbacks de acciones posibles. El padre mantiene la colección y decide su actualización.

No hay que aplicar siempre este patrón a cualquier componente del mundo. Un componente puede tener estado local propio, y muchos escenarios no necesitan callbacks. El criterio es que **la responsabilidad de modificar una colección debe quedar en el lugar que la posee o coordina**, mientras los hijos comunican intenciones mediante contratos claros.

Construir una evidencia visual de cada contrato

Con el navegador abierto, anota una prueba que demuestre cada dirección. Para padre→hijo, compara los valores de dos tarjetas diferentes. Para hijo→padre, pulsa Seleccionar y observa el texto externo. Para el reto de Quitar, verifica reducción del total. La compilación por sí sola no demuestra que la notificación llegue; el clic real sí permite observarlo.

No te conformes con una captura final que muestre solo dos productos: podría proceder de una colección inicial con dos elementos. La evidencia más convincente contiene la transición desde tres a dos causada por un botón concreto. Esto es especialmente relevante al ejecutar pruebas en navegador.

Una transferencia hacia M04 sin implementarla aún

Supongamos que en el módulo siguiente el padre dispone de un servicio HTTP para eliminar o actualizar un producto. La tarjeta podría seguir emitiendo el Id; el padre decidiría si invoca el servicio, muestra un error o refresca la lista. Esa posibilidad deriva de mantener el evento desacoplado del transporte. No añadas HTTP a P3.5: la finalidad es comprender el contrato antes de construir una operación remota.

Conclusión del laboratorio

Redacta en tu cuaderno dos frases distintas: «El padre entrega el producto por [Parameter]» y «El hijo notifica una intención por EventCallback». Después añade una tercera: «El padre aplica la modificación a su colección». Si las tres describen el comportamiento observado y puedes localizar las líneas correspondientes en los archivos, has comprendido la comunicación padre–hijo de este módulo.

P3.6. OnInitializedAsync, servicio en memoria y estados de carga

Objetivo y punto de partida

Hasta ahora, la página creaba directamente una colección de productos. En esta práctica aprenderemos a pedirlos a un **servicio en memoria asíncrono** mediante inyección de dependencias. El servicio no realiza HTTP ni utiliza base de datos: esa complejidad se introducirá de forma controlada en M04. El proyecto de trabajo es `M03_BLAZOR/PROYECTO/3.6/CursoWeb.Blazor/`.

La página inicial compila y muestra «Servicio pendiente de activar», pero todavía no llama al servicio porque el bloque de `OnInitializedAsync` permanece comentado. De esa manera puedes observar el estado anterior a la carga, activar un cambio identificado y comprobar su efecto en la interfaz.

3.6.1. Localizar el servicio y su registro

Abre `Services/IProductosMemoriaService.cs`. La interfaz declara:

```
Task<IReadOnlyList<Producto>> ObtenerAsync();
```

La implementación `Services/ProductosMemoriaService.cs` devuelve una colección de Cuaderno, Lápiz y Carpeta después de `await Task.Delay(700)`. Esta espera simula una operación asíncrona para poder ver la transición de estados; no representa acceso a una API externa.

En `Program.cs` comprueba el registro:

```
builder.Services.AddScoped<IProductosMemoriaService, ProductosMemoriaService>();
```

Y en `Components/Pages/Productos.razor` localiza:

```
@inject IProductosMemoriaService Catalogo
```

El componente solicita una dependencia por interfaz y la recibe del contenedor de ASP.NET Core. El alumno ya conoce el concepto de inyección de dependencias de otros ámbitos de .NET; ahora observa cómo participa en un componente Razor.

3.6.2. Ejecutar el estado inicial sin activar el método

Detén P3.5, abre 3.6, compila y ejecuta. Abre `/productos`. Debes encontrar el texto «Servicio pendiente de activar» y ninguna tarjeta. Observa los campos en `@code`:

```
private IReadOnlyList<Producto> productos = Array.Empty<Producto>();
private bool cargando = true;
private string estado = "Servicio pendiente de activar";
```

Este estado inicial no está «mal hecho»: la interfaz no llama aún al servicio y, por tanto, no tiene datos que representar. La finalidad del paso es comprobar que el alumno entiende la diferencia entre **un componente existente** y **un componente que ya cargó sus datos**.

3.6.3. Activar OnInitializedAsync

En `Components/Pages/Productos.razor` retira los delimitadores C# del bloque `PASO 3.6.2 INICIO / FIN`. El método activo debe quedar:

```
protected override async Task OnInitializedAsync()
{
    estado = "Cargando productos...";
    productos = await Catalogo.ObtenerAsync();
    cargando = false;
    estado = "Lista preparada";
}
```

Guarda, compila y recarga `/productos`. Primero debe aparecer «Cargando productos...»; cuando el servicio termine, debe mostrarse «Lista preparada» y tres tarjetas. Si no alcanzas a leer el primer mensaje por la velocidad del equipo, recuerda que la espera controlada se encuentra en el servicio y no en la vista.

El método no lo llama el alumno desde un botón: **Blazor lo invoca durante la inicialización del componente**. Tampoco es el handler `OnGet` de una Razor Page; esos nombres pertenecen a modelos diferentes. La propiedad `productos` se llena después del `await` y el estado de la interfaz cambia.

3.6.4. Interpretar el marcado condicional

En la misma página encontrarás:

```
<p id="estado-carga">@estado</p>
@if (!cargando && productos.Count > 0)
{
    <div class="tarjetas" id="tarjetas-productos">
        @foreach (var producto in productos)
        {
            <TarjetaProducto Producto="producto" />
        }
    </div>
}
```

La bandera `cargando` impide representar tarjetas mientras la operación no ha terminado. El valor de `productos.Count` determina si existe algo que mostrar. Esta separación permite razonar sobre estados reales y no sobre una pantalla que «a veces aparece y a veces no».

Caso negativo reversible: comenta temporalmente la asignación `cargando = false`, recompila y observa que, aun después de que el servicio devuelva objetos, la condición de la página no permite representar tarjetas. Restablece la asignación. El resultado ilustra que obtener datos y habilitar su representación son dos operaciones conectadas, pero distintas.

3.6.5. Comprender la elección de prerenderizado

En la cabecera de `Productos.razor` aparece:

```
@rendermode @(new InteractiveServerRenderMode(prerender: false))
```

La página desactiva el prerenderizado para facilitar una observación controlada de la carga y evitar una ejecución duplicada durante las fases de prerenderizado e inicialización interactiva. No lo interpretes como recomendación general para producción: el prerenderizado puede ser útil para presentar contenido antes de que la interactividad esté conectada.

En la teoría 3.6 se explica que `OnInitializedAsync` puede ejecutarse dos veces con prerenderizado. La práctica principal no implementa persistencia de estado de prerenderizado ni estrategias avanzadas de caché; basta comprender la razón de esta elección y saber localizarla.

Reto 3.6 — Lista vacía después de una carga correcta

Enunciado: muestra una indicación legible cuando el servicio ha terminado pero no hay productos. La pantalla debe distinguir «cargando» de «terminado con lista vacía».

Activa el comentario Razor `RETO 3.6 (ACTIVAR)`:

```
@if (!cargando && productos.Count == 0)
{
    <p id="sin-productos">No hay productos para mostrar.</p>
}
```

Para probar el caso negativo, abre `Services/ProductosMemoriaService.cs` y sustituye temporalmente el bloque de retorno de tres productos por:

```
return Array.Empty<Producto>();
```

Importante: reemplaza la instrucción `return` original, no añadas otra después de ella. Compila, abre `/productos` y observa que la carga termina con «Lista preparada» y el mensaje «No hay productos para mostrar», sin tarjetas. Esto no es un error de HTTP ni una excepción: el servicio ha devuelto una colección vacía válida.

Resultado positivo complementario: restaura la lista original del servicio y recarga. Deben volver a mostrarse tres tarjetas y el mensaje de

lista vacía no debe aparecer. La solución es una condición Razor dependiente de `cargando` y del número de elementos; no se necesita una API ni un nuevo componente.

3.6.7. Reto integrador — Simular un error de carga y recuperar el catálogo

Encargo. El catálogo ya obtiene sus productos de forma asíncrona desde un servicio en memoria. Amplía el ejercicio para que el usuario pueda simular un fallo, reconocer que la carga no terminó correctamente y **reintentar** hasta recuperar las tarjetas. Hay que conseguirlo con el servicio local existente; todavía no se utilizarán HttpClient, Web API, SQL ni persistencia.

Estado de partida. Trabaja en M03_BLAZOR/PROYECTO/3.6/CursoWeb.Blazor con los pasos de carga de P3.6 ya realizados. Al abrir /productos aparecen «Lista preparada» y tres tarjetas. Guarda una copia de este estado o utiliza la carpeta original limpia si has modificado el servicio para ensayar el resultado vacío. Un servicio que devuelve intencionadamente cero productos haría menos clara la comprobación de la recuperación tras error.

Paso 1 — Identificar el recorrido completo. Abre Services/IProductosMemoriaService.cs, Services/ProductosMemoriaService.cs y Components/Pages/Productos.razor. En los tres existe un bloque RETO INTEGRADOR 3.6.7 INICIO / FIN. El contrato anuncia una nueva operación asíncrona; la clase del servicio la implementa; el componente solicita esa operación y transforma el resultado en mensajes y tarjetas. No basta con activar solo el último archivo: la interfaz y la implementación forman parte de la misma compilación.

Paso 2 — Activar el contrato y la implementación. En IProductosMemoriaService.cs retira los delimitadores del bloque indicado para que declare:

```
Task<IReadOnlyList<Producto>> ObtenerConErrorAsync(bool simularError);
```

En ProductosMemoriaService.cs activa el método homónimo. El método espera brevemente, lanza InvalidOperationException cuando simularError es true y, cuando es false, obtiene el catálogo habitual mediante ObtenerAsync(). Antes de continuar, justifica el parámetro booleano: permite reproducir el error a voluntad y comprobar tanto el comportamiento fallido como el correcto sin hacer depender la práctica de un servidor remoto.

Paso 3 — Sustituir el método inicial, no duplicarlo. En la página Productos.razor activa el bloque C# del reto 3.6.7. Dentro encontrarás la nueva definición:

```
protected override Task OnInitializedAsync() => RecargarAsync();
```

Este método **reemplaza** al OnInitializedAsync que activaste en PASO 3.6.2. Elimina del archivo la definición anterior (puedes mantener las instrucciones históricas como comentarios si no duplican código activo). Si dejas dos métodos con la misma firma, el proyecto no compilará. El nuevo OnInitializedAsync llama a RecargarAsync para que la carga inicial y la carga solicitada con el botón compartan la misma lógica.

Paso 4 — Leer la lógica de recuperación. El método RecargarAsync establece cargando = true, limpia errorCarga, muestra «Cargando productos...» y deja una colección vacía mientras realiza la nueva petición al servicio. Dentro de try espera ObtenerConErrorAsync(simularError). Con éxito, asigna productos y cambia el estado a «Lista preparada». En catch (InvalidOperationException), genera un texto comprensible y cambia el estado a «Carga fallida». Finalmente, el bloque finally asigna cargando = false. ¿Qué sucedería si el método dejase la bandera en true después de capturar la excepción? Razona la respuesta mirando las condiciones Razor y el atributo disabled del botón.

Paso 5 — Activar los controles del navegador. Descomenta las tres líneas Razor preparadas junto a la lista: la casilla id="simular-error" enlazada a simularError; el botón id="recargar-productos" que ejecuta RecargarAsync y se deshabilita durante una carga; y el párrafo condicional id="error-carga" con role="alert". Esta última condición solo representa el mensaje cuando errorCarga no está vacío. Conserva también el texto existente id="estado-carga"; son elementos diferentes, uno indica el estado general y el otro explica un error concreto.

Paso 6 — Ensayo de éxito y fallo. Compila y abre /productos sin marcar la casilla. Espera a que aparezcan «Lista preparada» y tres tarjetas. Marca «Simular fallo», pulsa «Recargar catálogo» y espera a leer «Carga fallida». Debe verse «No se pudo cargar el catálogo de demostración.» y no deben aparecer tarjetas antiguas como si correspondieran a este nuevo intento. Comprueba que el botón vuelve a estar disponible: la excepción no debe dejar la interfaz atrapada en estado de carga.

Paso 7 — Recuperación sin reiniciar el sitio. Desmarca «Simular fallo», pulsa de nuevo «Recargar catálogo» y espera el resultado. El texto deberá volver a «Lista preparada», el mensaje de error desaparecerá y se mostrarán tres tarjetas. Esta recuperación ocurre en **el mismo componente**, después de un fallo conocido. No ha sido necesario editar datos, reiniciar dotnet ni ejecutar una API externa.

Paso 8 — Diferenciar tres síntomas. Repite el razonamiento con tres situaciones: (a) servicio correcto con tres elementos; (b) servicio correcto con lista vacía, como en el reto 3.6 anterior; (c) servicio que lanza la excepción simulada. En el caso (b) se terminó correctamente, por lo que corresponde un mensaje de vacío, no el mensaje de fallo. En el caso (c) la llamada no produjo una lista válida y se entra en catch. Explica por qué una bandera cargando no es suficiente para describir los tres resultados.

Incidencias que debes poder solucionar. Una declaración duplicada de OnInitializedAsync revela que no sustituiste el método previo. Un error «no contiene una definición para ObtenerConErrorAsync» obliga a revisar el contrato y la implementación. Un mensaje de fallo que permanece después del éxito obliga a revisar dónde se restablece errorCarga. Una pantalla sin tarjetas durante toda la ejecución exige comprobar si cargando pasa a false o si la llamada nunca termina. No soluciones estos síntomas añadiendo código JavaScript: la información relevante está en el ciclo de vida, el estado y el contrato de servicio.

Conclusión. Resume con tus palabras cuál es la responsabilidad de la página y cuál la del servicio. Identifica dónde sucede la espera, dónde se decide mostrar el mensaje de error y qué hace posible volver a intentarlo. Al terminar, el componente sabe recuperar una carga controlada; en M04 podrá aprender a usar una API real sin confundir un fallo de red con la mecánica elemental del estado de Blazor.

Cierre y transferencia a M04

Al terminar M03 debes poder explicar un recorrido completo de Blazor: una ruta selecciona un componente, un parámetro entrega datos a una tarjeta, un evento modifica estado, un callback notifica al padre, un servicio se obtiene mediante inyección y `OnInitializedAsync` carga una colección. Las pruebas realizadas deben relacionar cada comportamiento con sus archivos concretos; no basta con decir que los seis proyectos compilan.

En M04 sustituiremos o complementaremos el servicio en memoria por llamadas `HttpClient` a una API auxiliar suministrada por el curso. Precisamente por eso no adelantamos HTTP en P3.6: necesitamos que el alumno comprenda antes los componentes, su ciclo de vida y sus estados.

Laboratorio de profundización P3.6 — Carga asíncrona, resultado vacío y diagnóstico

El objetivo de esta ampliación es distinguir cuatro hechos que pueden resultar visualmente parecidos: el método de inicialización no está activado, el método está esperando, el servicio terminó con datos y el servicio terminó con una lista vacía. Todos pueden observarse con el mismo proyecto 3.6, sin API remota ni SQL. Trabaja sobre una copia docente limpia si has realizado varias modificaciones.

Caso A: el componente existe, pero no llama al servicio

Abre `/productos` con el bloque `PASO 3.6.2` todavía comentado. Debes leer «Servicio pendiente de activar». Comprueba que `Program.cs` registra correctamente `IProductosMemoriaService` y que el archivo de la página incluye `@inject`. Aunque esas piezas existan, el método `OnInitializedAsync` no se ejecuta porque permanece comentado. La dependencia está preparada, pero ninguna operación de carga ha sido programada en la instancia.

Este caso demuestra que registrar un servicio no hace que todos los componentes lo llamen automáticamente. La llamada real se encuentra en el método de inicialización que activaremos en el siguiente estado. No confundas **disponer de una dependencia** con **invocarla**.

Caso B: el servicio recibe la llamada y la pantalla muestra espera

Activa `OnInitializedAsync` y ejecuta de nuevo. El método asigna `estado = "Cargando productos..."` antes del `await`. Dado que la página utiliza Interactive Server con `prerender: false`, podemos observar el momento en que el componente prepara la espera. El servicio contiene `Task.Delay(700)` como latencia simulada. No utilizamos un temporizador de presentación ni introducimos una segunda tarea innecesaria; la espera está en la operación de obtención de datos.

En el navegador, intenta observar la transición de «Cargando productos...» a «Lista preparada». La duración exacta puede variar, porque compilar y

abrir recursos también consume tiempo. El resultado esencial no es medir milisegundos, sino distinguir los estados y comprender qué código los produce.

Caso C: los datos llegan, pero no hay tarjetas

Para reproducir un error de presentación, comenta temporalmente `cargando = false` después de la llamada al servicio. Aunque `productos` reciba la lista, la condición `!cargando && productos.Count > 0` no permite representarla. Esto explica un informe frecuente: «el servicio parece devolver productos, pero la página está vacía». La causa no tiene que ser el transporte; puede encontrarse en una bandera que el componente utiliza para decidir qué dibujar.

Restablece la asignación. Ahora las tarjetas deben aparecer cuando termine la carga. La solución razonada consiste en mantener coherentes los datos obtenidos y el indicador de finalización; no es mover las tarjetas a otro layout ni eliminar la condición para ocultar el problema.

Caso D: el servicio termina correctamente con cero productos

Activa el reto del mensaje `sin-productos` y sustituye temporalmente el retorno del servicio por `Array.Empty<Producto>()`. Conserva la espera para observar la transición. La carga terminará y la lista quedará vacía, pero la interfaz ya no debería quedarse en un estado ambiguo: debe informar «No hay productos para mostrar».

Este escenario es distinto del anterior. En C la condición estaba bloqueada por una bandera que nunca se actualizó; en D la bandera indica finalización y la colección contiene cero elementos de forma legítima. **Resultado vacío no equivale a fallo del servicio**. El alumno debe demostrarlo observando el estado «Lista preparada» junto al mensaje de ausencia de datos.

Caso E: el servicio no está registrado

Como ejercicio conceptual, imagina que se eliminara `AddScoped<IProductosMemoriaService, ProductosMemoriaService>()` de `Program.cs` mientras el componente sigue solicitando `@inject IProductosMemoriaService Catalogo`. La inyección de dependencias ya no tendría una implementación disponible. El diagnóstico debería dirigirse al registro del contenedor, no a la sintaxis del `foreach` ni a los atributos HTML.

No recomendamos romper el proyecto como actividad obligatoria si el alumno todavía está aprendiendo a cargar datos. Basta con localizar las dos piezas y explicar por qué deben corresponderse. La teoría 3.6 detalla esta relación y su conexión con la inyección utilizada anteriormente en Web API.

Caso F: inicialización duplicada por prerenderizado

En una aplicación Blazor Web App con prerenderizado, `OnInitializedAsync` puede ejecutarse durante la representación inicial en servidor y otra vez cuando se inicia el componente interactivo. Por eso el proyecto 3.6 declara `prerender: false`: permite centrar la práctica en una transición controlada. Esta decisión no debe convertirse en una regla universal para producción, donde el prerenderizado puede ser valioso.

Como transferencia, imagina que el servicio realizara una operación no idempotente en lugar de devolver datos. Dos inicializaciones podrían duplicar efectos. ¿Qué deberíamos investigar? La estrategia de prerenderizado, la ubicación de la operación y cómo conservar o compartir resultados. No necesitas implementar esas soluciones avanzadas en M03; basta comprender por qué sería importante analizar el ciclo de vida.

Resolver un reporte de error paso a paso

Un alumno comunica: «La lista se queda en blanco». Comienza preguntando si hay un mensaje visible. Si dice «Servicio pendiente de activar», revisa que el bloque de `OnInitializedAsync` esté activo. Si dice

«Cargando productos...» durante mucho tiempo, revisa si el método termina y si hay errores durante `ObtenerAsync`. Si dice «Lista preparada» pero no aparecen tarjetas, comprueba `cargando` y `productos.Count`. Si dice «No hay productos para mostrar», quizá la lista esté vacía como resultado válido del servicio o se haya dejado activa la variante del reto.

La observación de estados reduce diagnósticos erróneos. No toda pantalla sin tarjetas requiere un cambio de `@rendermode` o un nuevo servidor HTTP. En este proyecto la red no interviene como origen de datos, así que no debemos inventar un problema de API para explicar una lista vacía.

Respuesta razonada esperada

Elabora una explicación que conecte `@inject`, el registro `AddScoped`, `OnInitializedAsync`, `await Catalogo.ObtenerAsync()`, `cargando` y el marcado condicional. Debes poder señalar el archivo responsable de cada etapa: `Program.cs` configura el contenedor, `IProductosMemoriaService.cs` declara el contrato, `ProductosMemoriaService.cs` suministra datos y `Productos.razor` administra la presentación.

Si puedes diferenciar los resultados sin API, estarás preparado para estudiar M04: allí los mismos estados de interfaz deberán convivir con errores HTTP, deserialización JSON y respuestas remotas. No se trata de repetir desde cero la programación de componentes, sino de cambiar de manera controlada la fuente de datos.

Autoevaluación integradora

Al final del módulo, abre P3.6 y describe cinco diferencias entre la aplicación MVC A y la aplicación Blazor B sin recurrir a generalidades. Señala el lugar donde se configura Interactive Server; cómo se declara una ruta `@page`; cómo una tarjeta recibe un producto; cómo un método de clic actualiza un texto; y cómo el servicio obtiene datos sin que la página sepa dónde se construye su colección.

Si puedes relacionar estos cinco aspectos con código real, explicar un caso positivo y negativo y distinguir estado de componente de persistencia, habrás alcanzado el objetivo de la jornada. No hay que construir todavía un CRUD nuevo ni desplegar infraestructura adicional.

Taller complementario — Ocho incidentes reproducibles y sus soluciones

Este taller amplía el material de consulta; **no añade ocho actividades obligatorias a las seis horas de clase**. Permite repetir una situación problemática, observar qué falla y recuperarla con un cambio controlado. Cada caso debe realizarse en su carpeta P3.N, partiendo del estado activado que indiquen las instrucciones anteriores. Al terminar, restaura los cambios experimentales. No necesitas otra aplicación, una API, SQL Server ni bibliotecas nuevas.

Incidente A — El método existe, pero el clic no responde (P3.1)

Una persona abre `/estatica`, observa el código `@onclick="Incrementar"` y concluye que el método C# está mal escrito porque el contador no aumenta. Antes de modificar nada, compáralo con `/interactiva`: ambos contienen un botón que invoca un método semejante, pero solo la segunda página declara `@rendermode InteractiveServer`.

Reproducción: ejecuta el proyecto 3.1, abre `/estatica`, pulsa dos veces y comprueba que el valor permanece en cero. Abre `/interactiva`, espera la conexión y pulsa dos veces. Ahora debe aparecer el valor dos. Si ambos botones compilan, su diferencia principal no es aritmética. Comprueba `Program.cs` y `App.razor` para localizar los servicios, endpoints y recursos del host, pero no añadas otro controlador.

Experimento reversible: quita temporalmente la directiva de modo de `Interactiva.razor`, recompila y repite el clic. La página deja de responder como componente interactivo. Restaura la directiva y repite la prueba positiva. El

resultado observable identifica una condición concreta del modelo de ejecución.

Explicación causal: una respuesta HTML puede contener el valor inicial de un campo C# sin disponer del mecanismo de eventos que permite modificarlo después del clic. Compilar y descargar la página no sustituyen la prueba de interacción en navegador. La solución no es crear un POST MVC ni añadir JavaScript manualmente: es comprender el modo declarado y su infraestructura.

Incidente B — La página existe, pero no aparece en el menú (P3.2)

Un alumno escribe `/productos` y ve Cuaderno, Lápiz y Carpeta. Después regresa al inicio y no encuentra un enlace Productos. Supone que la ruta está mal registrada. Antes de editar `@page`, observa que la dirección directa ya ha demostrado la existencia del componente.

Reproducción: abre el proyecto 3.2 con `PASO 3.2.2` aún comentado. Visita la ruta directa y después el inicio. Abre `Components/Layout/MainLayout.razor` y activa el bloque `NavLink href="/productos"`. Compila y comprueba que el enlace aparece y conduce a la misma ruta que antes funcionaba escribiéndola directamente.

Control negativo: modifica temporalmente solo el `href` hacia una ruta inexistente. El menú puede mostrar el enlace, pero su destino no corresponderá a una página. Restablece `/productos` al terminar. Esta prueba separa la existencia visual de un enlace de la existencia del endpoint que representa su destino.

Explicación causal: `@page` declara la ruta; el layout ofrece navegación. Si la ruta funciona pero no tiene enlace, revisa el layout. Si el enlace existe pero el destino muestra contenido de NotFound, revisa las rutas. Si la ruta funciona pero faltan productos, revisa los datos de la página. Cambiar el nombre del record Producto no resuelve ninguno de los dos primeros problemas.

Incidente C — Tres tarjetas, pero las tres contienen Cuaderno (P3.3)

Tras sustituir la lista por TarjetaProducto, el alumno ve tres artículos. Sin embargo, los tres tienen el mismo nombre y referencia. Este escenario distingue un contrato válido de un valor **semánticamente equivocado** transmitido por el padre.

Reproducción: utiliza la carpeta 3.3 con las tarjetas activadas y la lista anterior desactivada. Comprueba primero que cada tarjeta recibe un Id distinto. Cambia temporalmente la expresión `Producto="producto"` del padre por `Producto="productos[0]"`. Compila y observa la pantalla: hay tres instancias de componente, pero todas representan el primer objeto de la colección.

Inspección: localiza en el navegador el atributo `data-producto-id` de cada tarjeta. Si todos contienen el primer identificador, no basta afirmar que el bucle funciona porque se ven tres tarjetas. El bucle se ejecuta, pero transmite un valor equivocado al hijo en cada iteración.

Corrección: restablece `Producto="producto"`. La tarjeta declara una propiedad `[Parameter]` que recibe lo que el padre le proporciona; no puede adivinar qué objeto debería haber recibido. Comprueba de nuevo nombres, precios e identificadores. La prueba funcional necesita verificar cantidad **e identidad**, no únicamente cantidad de HTML generado.

Incidente D — El botón Añadir no compila después de activarlo (P3.4)

Un alumno descomenta la línea `<button id="agregar" @onclick="Agregar">` y ejecuta `dotnet build`. Aparece un error relativo al método de evento o al campo `nuevoNombre`. Este caso enseña que los pasos numerados tienen dependencias de código, no solo fragmentos visuales.

Preparación: abre un estado inicial limpio de la carpeta 3.4. Localiza tres grupos marcados: `PASO 3.4.2`, `PASO 3.4.3` y `PASO 3.4.4`. Cada uno incluye elementos Razor y miembros C#. Debes activar cada grupo de forma completa antes de comprobarlo, sin copiar métodos de proyectos posteriores.

Secuencia correcta: activa primero el botón Seleccionar y el método `Seleccionar(int)`. Compila y observa su efecto. Activa después el input `@bind="nuevoNombre"` y el campo C# correspondiente. Vuelve a compilar. Finalmente activa el botón Añadir y **todo** su bloque C#, incluido `siguienteId` y `Agregar()`. El método utiliza ambos datos, por lo que no puede compilar correctamente si alguno falta.

Prueba visible: escribe Regla y pulsa Añadir. Debe aparecer una cuarta tarjeta. Prueba una entrada vacía: no se incorpora otro objeto. El resultado no requiere una acción HTTP POST ni un DbContext. Si el botón sigue sin responder pese a compilar, comprueba modo interactivo, enlace de datos y manejador real, distinguiendo sus responsabilidades.

Incidente E — La cuarta tarjeta desaparece después de recargar (P3.4)

En 3.4, un alumno añade Regla y cuenta cuatro artículos. Recarga por completo y vuelve a encontrar solo tres. Lo interpreta como un fallo de persistencia. La comparación con M02 permite demostrar por qué ese criterio no es apropiado para el estado local de esta práctica.

Observación: abre el método `Agregar` y busca una llamada a `productos.Add`. Después busca una operación de almacenamiento persistente, como `SaveChangesAsync` o un servicio que comunique el cambio a una API. El proyecto no contiene esa infraestructura, porque todavía estamos aprendiendo eventos y estado del componente.

Experimento: selecciona Lápiz, añade Regla y observa que ambos cambios se reflejan. Recarga la página para que se cree una nueva instancia. Los campos vuelven a sus valores iniciales. No es evidencia de que `@onclick` estuviera roto; el manejador ya produjo el cambio requerido durante la sesión.

Explicación causal: en M02 el resultado de un POST podía almacenarse en LocalDB y comprobarse después de reiniciar la aplicación. En P3.4 la lista existe dentro de una instancia interactiva. Su criterio de éxito es la

actualización visible en esa instancia, no una promesa de persistencia que no se ha implementado. No utilices una lista estática global para ocultar la diferencia.

Incidente F — La activación de callbacks produce tarjetas duplicadas (P3.5)

La página P3.5 entrega tres variantes de etiqueta `TarjetaProducto`: sin callback, con selección y con selección y eliminación. Si activas una nueva variante pero conservas la línea anterior, cada vuelta del `foreach` representa dos tarjetas, aunque la colección C# siga teniendo tres productos.

Reproducción: sobre una copia del proyecto 3.5, deja activa la variante inicial y activa además la variante con `AlSeleccionar`. Comprueba cuántos artículos HTML aparecen. El error se reproduce sin modificar la lista ni el modelo. No lo corrijas borrando productos del campo `productos`: eso ocultaría la causa y cambiaría los datos que otros pasos esperan.

Corrección: mantén una sola variante activa dentro del `foreach`. Comprueba que el hijo declara los parámetros que esa variante utiliza y que el padre dispone de los métodos receptores. Ahora pulsa Seleccionar sobre Lápiz y verifica el texto del padre. Si continúas con el reto, sustituye otra vez la etiqueta por la variante con `AlQuitar` y comprueba que desaparece un solo producto.

Explicación causal: duplicar instancias de componente es un problema de composición, no una consecuencia automática de EventCallback. El callback transmite intenciones; el número de tarjetas depende de las etiquetas activas dentro del bucle.

Incidente G — Se quitó la tarjeta, pero continúa seleccionada (P3.5)

El reto de eliminación produce tres cambios relacionados: el producto desaparece, el total disminuye y el texto de selección vuelve a «ninguno». Si falta el último, el callback puede haber funcionado aunque el estado de la interfaz quede incoherente.

Experimento: activa el reto y selecciona Lápiz. En `Quitar(int id)` comenta temporalmente solo `seleccion = "ninguno"`. Pulsa Quitar. La tarjeta puede desaparecer y el total reducirse, mientras la cabecera sigue mencionando el producto eliminado. El problema no está necesariamente en `AlQuitar.InvokeAsync`: el padre recibió el Id y eliminó el objeto.

Corrección: restaura la asignación que limpia la selección y repite. El estado visible debe resultar coherente. Esta prueba enseña a separar **comunicación correcta entre componentes** de **reglas correctas de actualización del estado**. El componente hijo no debería acceder por su cuenta al campo de selección del padre; simplemente comunica la intención de quitar un producto.

Transferencia: en una aplicación mayor, el padre podría solicitar confirmación antes de quitar. El hijo continuaría enviando un Id. No es necesario incorporar esa funcionalidad en M03 para apreciar la ventaja de un contrato de callback pequeño y explícito.

Incidente H — El servicio devuelve productos, pero no se dibujan (P3.6)

El componente de P3.6 obtiene tres objetos mediante `Catalogo.ObtenerAsync()` y los asigna a `productos`. Sin embargo, el marcado requiere además que `cargando` sea falso. Es posible que existan datos correctos y que la representación todavía no esté habilitada por el propio estado de presentación.

Reproducción: activa `OnInitializedAsync` y comprueba la carga normal. Después comenta temporalmente `cargando = false`. El servicio puede completar su tarea, pero el bloque `@if (!cargando && productos.Count > 0)` no representa tarjetas. No se trata de un fallo de CSS ni de una API remota: la condición está esperando una bandera que nunca cambia.

Corrección: restaura la asignación y comprueba que las tarjetas aparecen. Activa ahora el reto de lista vacía y cambia el retorno del servicio por `Array.Empty<Producto>()`. El resultado correcto es distinto: la carga

termina y la pantalla muestra «No hay productos para mostrar» porque no existen elementos. No debe confundirse con esperar indefinidamente.

Explicación causal: la secuencia completa es registro del servicio, inyección, inicialización, espera asíncrona, asignación de la lista, actualización de `cargando` y representación. La ausencia de tarjetas puede indicar fallos diferentes según cuál de esos pasos no se haya completado. En M04 añadiremos fallos HTTP, pero todavía no son una causa posible del servicio local de P3.6.

Recuperación del estado de trabajo

Tras cada experimento, revierte únicamente el cambio utilizado para provocarlo. Si tienes dudas, recupera el archivo de la carpeta de la práctica que estás realizando, no el de otro punto. El anexo de archivos canónicos de este mismo cuaderno permite comparar el código con el estado inicial versionado.

Al explicar un incidente, escribe tres elementos: **síntoma observado**, **hipótesis sobre la responsabilidad que falla** y **prueba que distingue esa hipótesis de otras**. Esta secuencia ayuda más que modificar varios archivos a la vez. El objetivo es que el alumno pueda razonar sobre su propia aplicación y no dependa de reconstruir la solución completa cada vez que un botón o una página no se comporta como espera.

Anexo práctico — Archivos canónicos de cada estado inicial

Este anexo reproduce íntegramente los archivos específicos que intervienen en las transiciones del módulo, extraídos de la misma rama del proyecto. No sustituye las instrucciones anteriores: sirve para consultar el **estado inicial reproducible**, recuperar un bloque comentado o comparar tu solución con el archivo entregado. Los archivos comunes que no cambian entre los seis estados se omiten aquí para evitar repetir boilerplate. Cada fragmento conserva su ruta exacta, y los cambios posteriores se efectúan siguiendo los pasos 3.1–3.6.

Archivos específicos de P3.1

Program.cs

Ruta exacta: `M03_BLAZOR/PROYECTO/3.1/CursoWeb.Blazor/Program.cs`.

Arranque del host, servicios de Razor Components y mapeo de Interactive Server. Compara ambos registros con las directivas de las dos páginas.

```
using CursoWeb.Blazor.Components;

var builder = WebApplication.CreateBuilder(args);
// Servicios necesarios para componentes Razor y circuitos Interactive Server.
builder.Services.AddRazorComponents().AddInteractiveServerComponents();

var app = builder.Build();
if (!app.Environment.IsDevelopment()) app.UseExceptionHandler("/error");
app.UseStaticFiles();
app.UseAntiforgery();
// El modo Interactive Server necesita tanto servicios como endpoint.
app.MapRazorComponents<App>().AddInteractiveServerRenderMode();
app.Run();
public partial class Program { }
```

Components/Pages/Estatica.razor

Ruta exacta: `M03_BLAZOR/PROYECTO/3.1/CursoWeb.Blazor/Components/Pages/Estatica.razor`.

Primera mitad de la comparación: el método existe, pero la página no declara un modo interactivo.

```
@page "/estatica"
<PageTitle>SSR estático</PageTitle>
<h1>P3.1 | Componente con SSR estático</h1>
<p>El servidor envía HTML. Este ejemplo no declara un modo de renderizado
interactivo.</p>
<p id="contador-estatico">Pulsaciones: @contador</p>
<button id="boton-estatico" @onclick="Incrementar">Pulsar (sin interacción)</
button>
<p class="mensaje">Aunque el marcado contenga @@onclick, sin renderizado
interactivo no se ejecuta el evento C# tras pulsar.</p>
@code {
    private int contador;
    private void Incrementar() => contador++;
}
```

Components/Pages/Interactiva.razor

Ruta exacta: `M03_BLAZOR/PROYECTO/3.1/CursoWeb.Blazor/Components/Pages/Interactiva.razor`.

Segunda mitad: la directiva InteractiveServer habilita el circuito y contiene la variante de incremento doble del reto.

```
@page "/interactiva"
@rendermode InteractiveServer
<PageTitle>Interactive Server</PageTitle>
<h1>P3.1 | Componente interactivo en servidor</h1>
<p>El evento se procesa en el circuito de Blazor y la pantalla se actualiza sin
navegación HTTP.</p>
<p id="contador-interactivo">Pulsaciones: @contador</p>
<button id="boton-interactivo" @onclick="Incrementar">Pulsar (interactivo)</
button>
@code {
    private int contador;
    private void Incrementar() => contador++;
    // RETO 3.1: sustituye la línea anterior por esta variante (no mantener
ambas):
    // private void Incrementar() => contador += 2;
}
```

Archivos específicos de P3.2

Components/Layout/MainLayout.razor

Ruta exacta: `M03_BLAZOR/PROYECTO/3.2/CursoWeb.Blazor/Components/Layout/MainLayout.razor`.

El enlace Productos está preparado en un comentario; su activación permite navegar a una ruta que ya existe.

```
@inherits LayoutComponentBase
<header><strong>Curso ASP.NET Core · Blazor B</strong>
<nav>
    <NavLink href="/" Match="NavLinkMatch.All">Inicio</NavLink>
    <NavLink href="/estatica">SSR estático</NavLink>
    <NavLink href="/interactiva">SSR interactivo</NavLink>
    @* PASO 3.2.2 (ACTIVAR): quita las marcas de comentario de este NavLink. *@
    @* <NavLink href="/productos">Productos</NavLink> *@
</nav></header>
<main>@Body</main>
```

Components/Pages/Productos.razor

Ruta exacta: `M03_BLAZOR/PROYECTO/3.2/CursoWeb.Blazor/Components/Pages/Productos.razor`.

Colección inicial y bucle Razor. El ejercicio de cuarta fila está identificado junto a los datos a modificar.

```
@page "/productos"
@rendermode InteractiveServer
<PageTitle>Productos</PageTitle>
<h1>P3.2 | Página Productos</h1>
<p>Catálogo de ejemplo en memoria. Todavía no consumimos una API.</p>
<ul id="lista-productos">
    @foreach (var producto in productos)
    {
        <li>@producto.Nombre — @producto.Precio.ToString("0.00") €</li>
    }
</ul>
@code {
    private readonly List<Producto> productos =
    [
        new(1, "Cuaderno", 4.50m),
        new(2, "Lápiz", 1.20m),
        new(3, "Carpeta", 3.75m)
        // APLICACIÓN 3.2: añade una coma tras Carpeta y activa esta cuarta fila.
        // , new(4, "Regla", 2.50m)
    ];
}
```

Archivos específicos de P3.3

Components/TarjetaProducto.razor

Ruta exacta: `M03_BLAZOR/PROYECTO/3.3/CursoWeb.Blazor/Components/TarjetaProducto.razor`.

Contrato [Parameter] y representación visual de un objeto. Contiene la variante condicional del reto.

```
<article class="tarjeta" data-producto-id="@Producto.Id">
    <h2>@Producto.Nombre</h2>
    <p>Precio: @Producto.Precio.ToString("0.00") €</p>
    <small>Referencia @Producto.Id</small>
    @* RETO 3.3: eliminar los delimitadores del bloque siguiente. *@
    @*
    @if (Producto.Precio > 4m)
```

```
    {
        <p class="mensaje">Precio superior a 4 euros</p>
    }
    *@
</article>
@code {
    [Parameter, EditorRequired] public Producto Producto { get; set; } = default!;
}
```

Components/Pages/Productos.razor

Ruta exacta: `M03_BLAZOR/PROYECTO/3.3/CursoWeb.Blazor/Components/Pages/Productos.razor`.

Muestra las dos representaciones alternativas del padre: lista HTML inicial y tarjetas comentadas; debes sustituir una por otra.

```
@page "/productos"
@rendermode InteractiveServer
<PageTitle>Productos</PageTitle>
<h1>P3.3 | Composición y parámetros</h1>
<p>Tres productos en memoria. Cada tarjeta es una instancia del mismo
componente.</p>
@* PASO 3.3.2: el siguiente listado ya funciona en el estado inicial. *@
<ul id="lista-productos">
    @foreach (var producto in productos)
    {
        <li>@producto.Nombre — @producto.Precio.ToString("0.00") €</li>
    }
</ul>
@* PASO 3.3.3 (ACTIVAR): comenta el bloque <ul> anterior y quita los
   delimitadores del bloque siguiente, que pasa cada objeto mediante [Parameter].
*@
@*
<div class="tarjetas" id="tarjetas-productos">
    @foreach (var producto in productos)
    {
        <TarjetaProducto Producto="producto" />
    }
</div>
*@
@code {
    private readonly List<Producto> productos =
    [
        new(1, "Cuaderno", 4.50m),
        new(2, "Lápiz", 1.20m),
        new(3, "Carpeta", 3.75m)
```

```
    ];
}
```

Archivos específicos de P3.4

Components/Pages/Productos.razor

Ruta exacta: `M03_BLAZOR/PROYECTO/3.4/CursoWeb.Blazor/Components/Pages/Productos.razor`.

Contiene cada transición numerada: selección, binding, alta y reto. Observa que los campos y métodos todavía no activados no participan en la compilación inicial.

```
@page "/productos"
@rendermode InteractiveServer
<PageTitle>Eventos</PageTitle>
<h1>P3.4 | Estado y eventos</h1>
<p>Los datos existen únicamente durante la sesión interactiva de este
componente.</p>
<p id="seleccion">Seleccionado: @seleccion</p>
<div class="tarjetas" id="tarjetas-productos">
    @foreach (var producto in productos)
    {
        <article class="tarjeta" data-producto-id="@producto.Id">
            <strong>@producto.Nombre</strong>
            <p>@producto.Precio.ToString("0.00") €</p>
            @* PASO 3.4.2 (ACTIVAR): descomenta este botón al activar Seleccionar.
*@
            @* <button @onclick="() => Seleccionar(producto.Id)">Seleccionar</
button> *@
        </article>
    }
</div>
@* PASO 3.4.3 (ACTIVAR): descomenta el input tras activar el campo nuevoNombre. *@
@* <label>Nombre nuevo: <input id="nombre-nuevo" @bind="nuevoNombre" /></label> *@
@* PASO 3.4.4 (ACTIVAR): descomenta el botón tras activar Agregar. *@
@* <button id="agregar" @onclick="Agregar">Añadir producto</button> *@
@* RETO 3.4 (ACTIVAR): botón para deshacer la selección. *@
@* <button id="limpiar" @onclick="Limpiar">Limpiar selección</button> *@
@code {
    private readonly List<Producto> productos =
    [
        new(1, "Cuaderno", 4.50m),
        new(2, "Lápiz", 1.20m),
        new(3, "Carpeta", 3.75m)
    ];
```

```
    private string seleccion = "ninguno";

    /* PASO 3.4.2 INICIO
    private void Seleccionar(int id)
    {
        seleccion = productos.First(p => p.Id == id).Nombre;
    }
    PASO 3.4.2 FIN */

    /* PASO 3.4.3 INICIO
    private string nuevoNombre = string.Empty;
    PASO 3.4.3 FIN */

    /* PASO 3.4.4 INICIO
    private int siguienteId = 4;
    private void Agregar()
    {
        if (string.IsNullOrWhiteSpace(nuevoNombre))
            return;
        productos.Add(new Producto(siguienteId++, nuevoNombre.Trim(), 5m));
        nuevoNombre = string.Empty;
    }
    PASO 3.4.4 FIN */

    /* RETO 3.4 INICIO
    private void Limpiar() => seleccion = "ninguno";
    RETO 3.4 FIN */
}
```

Archivos específicos de P3.5

Components/TarjetaProducto.razor

Ruta exacta: `M03_BLAZOR/PROYECTO/3.5/CursoWeb.Blazor/Components/TarjetaProducto.razor`.

Parámetros y callbacks del hijo, incluidos los métodos que emiten una intención tipada.

```
<article class="tarjeta" data-producto-id="@Producto.Id">
    <h2>@Producto.Nombre</h2>
    <p>Precio: @Producto.Precio.ToString("0.00") €</p>
    <small>Referencia @Producto.Id</small>
    @* PASO 3.5.2 (ACTIVAR) al declarar AlSeleccionar: *@
    @* <button @onclick="NotificarSeleccion">Seleccionar</button> *@
    @* RETO 3.5 (ACTIVAR) al declarar AlQuitar: *@
    @* <button @onclick="NotificarQuitar">Quitar</button> *@
</article>
```

```
@code {
    [Parameter, EditorRequired] public Producto Producto { get; set; } = default!;

    /* PASO 3.5.2 INICIO
    [Parameter] public EventCallback<int> AlSeleccionar { get; set; }
    private Task NotificarSeleccion() => AlSeleccionar.InvokeAsync(Producto.Id);
    PASO 3.5.2 FIN */

    /* RETO 3.5 INICIO
    [Parameter] public EventCallback<int> AlQuitar { get; set; }
    private Task NotificarQuitar() => AlQuitar.InvokeAsync(Producto.Id);
    RETO 3.5 FIN */
}
```

Components/Pages/Productos.razor

Ruta exacta: `M03_BLAZOR/PROYECTO/3.5/CursoWeb.Blazor/Components/Pages/Productos.razor`.

Tres variantes de tarjeta en el padre, métodos de recepción y lista propietaria del estado; nunca dejes activas las tres variantes simultáneamente.

```
@page "/productos"
@rendermode InteractiveServer
<PageTitle>Comunicación</PageTitle>
<h1>P3.5 | Comunicación padre–hijo</h1>
<p id="seleccion">Seleccionado: @seleccion</p>
<div class="tarjetas" id="tarjetas-productos">
    @foreach (var producto in productos)
    {
        @* PASO 3.5.2: al activar AlSeleccionar en la tarjeta,
           reemplaza esta línea por la variante siguiente. *@
        <TarjetaProducto Producto="producto" />
        @* <TarjetaProducto Producto="producto" AlSeleccionar="Seleccionar" /> *@
        @* RETO 3.5: reemplaza por esta tercera variante al activar AlQuitar: *@
        @* <TarjetaProducto Producto="producto" AlSeleccionar="Seleccionar"
AlQuitar="Quitar" /> *@
    }
</div>
<p id="cantidad">Total: @productos.Count</p>
@code {
    private readonly List<Producto> productos =
    [
        new(1, "Cuaderno", 4.50m),
        new(2, "Lápiz", 1.20m),
        new(3, "Carpeta", 3.75m)
```

```
    ];
    private string seleccion = "ninguno";

    /* PASO 3.5.2 INICIO
    private void Seleccionar(int id)
    {
        seleccion = productos.First(p => p.Id == id).Nombre;
    }
    PASO 3.5.2 FIN */

    /* RETO 3.5 INICIO
    private void Quitar(int id)
    {
        productos.RemoveAll(p => p.Id == id);
        seleccion = "ninguno";
    }
    RETO 3.5 FIN */
}
```

Archivos específicos de P3.6

Program.cs

Ruta exacta: `M03_BLAZOR/PROYECTO/3.6/CursoWeb.Blazor/Program.cs`.

Registro del contrato IProductosMemoriaService y su implementación, separado de los archivos de presentación.

```
using CursoWeb.Blazor.Components;
using CursoWeb.Blazor.Services;

var builder = WebApplication.CreateBuilder(args);
// Servicios necesarios para componentes Razor y circuitos Interactive Server.
builder.Services.AddRazorComponents().AddInteractiveServerComponents();
// PASO 3.6.1: inyección por interfaz. No existe HttpClient ni API aún.
builder.Services.AddScoped<IProductosMemoriaService, ProductosMemoriaService>();
var app = builder.Build();
if (!app.Environment.IsDevelopment()) app.UseExceptionHandler("/error");
app.UseStaticFiles();
app.UseAntiforgery();
// El modo Interactive Server necesita tanto servicios como endpoint.
app.MapRazorComponents<App>().AddInteractiveServerRenderMode();
app.Run();
public partial class Program { }
```

Services/IProductosMemoriaService.cs

Ruta exacta: `M03_BLAZOR/PROYECTO/3.6/CursoWeb.Blazor/Services/IProductosMemoriaService.cs`.

Interfaz asíncrona que conoce el componente; no incorpora llamadas HTTP.

```
using CursoWeb.Blazor.Models;
namespace CursoWeb.Blazor.Services;
public interface IProductosMemoriaService
{
    Task<IReadOnlyList<Producto>> ObtenerAsync();
}
```

Services/ProductosMemoriaService.cs

Ruta exacta: `M03_BLAZOR/PROYECTO/3.6/CursoWeb.Blazor/Services/ProductosMemoriaService.cs`.

Implementación controlada, productos ficticios y retorno alternativo de lista vacía para el reto.

```
using CursoWeb.Blazor.Models;
namespace CursoWeb.Blazor.Services;
/// <summary>Servicio de laboratorio sin HTTP, EF Core ni base externa.</summary>
public sealed class ProductosMemoriaService : IProductosMemoriaService
{
    public async Task<IReadOnlyList<Producto>> ObtenerAsync()
    {
        await Task.Delay(700);
        // RETO 3.6 (alternativa): sustituye el bloque return siguiente por:
        // return Array.Empty<Producto>();
        return
        [
            new Producto(1, "Cuaderno", 4.50m),
            new Producto(2, "Lápiz", 1.20m),
            new Producto(3, "Carpeta", 3.75m)
        ];
    }
}
```

Components/Pages/Productos.razor

Ruta exacta: `M03_BLAZOR/PROYECTO/3.6/CursoWeb.Blazor/Components/Pages/Productos.razor`.

Inyección, ciclo de inicialización preparado, bandera de carga, estados de presentación y mensaje opcional de lista vacía.

```
@page "/productos"
@rendermode @(new InteractiveServerRenderMode(prerender: false))
@inject IProductosMemoriaService Catalogo
<PageTitle>Carga de productos</PageTitle>
<h1>P3.6 | OnInitializedAsync</h1>
<p>El servicio de productos usa datos en memoria y simula una operación
asíncrona.</p>
<p id="estado-carga">@estado</p>
@if (!cargando && productos.Count > 0)
{
    <div class="tarjetas" id="tarjetas-productos">
        @foreach (var producto in productos)
        {
            <TarjetaProducto Producto="producto" />
        }
    </div>
}
@* RETO 3.6 (ACTIVAR): mostrar un mensaje cuando no existen elementos.
   La condición no se cumple con los tres productos de ejemplo;
   prueba el reto retornando una lista vacía desde el servicio. *@
@* @if (!cargando && productos.Count == 0) { <p id="sin-productos">No hay
productos para mostrar.</p> } *@

@code {
    private IReadOnlyList<Producto> productos = Array.Empty<Producto>();
    private bool cargando = true;
    private string estado = "Servicio pendiente de activar";

    /* PASO 3.6.2 INICIO
    protected override async Task OnInitializedAsync()
    {
        estado = "Cargando productos...";
        productos = await Catalogo.ObtenerAsync();
        cargando = false;
        estado = "Lista preparada";
    }
    PASO 3.6.2 FIN */
}
```

Cómo utilizar este anexo sin destruir el estado de trabajo

Cuando un ejercicio deje de compilar, compara únicamente el archivo que modificaste con el estado inicial mostrado para su punto. Comprueba si conservaste ambos delimitadores de comentario o si retiraste solo uno. Para una sustitución de componentes en un bucle, cuenta las variantes activas. Si trabajas en una carpeta posterior, no copies indiscriminadamente la versión del punto anterior: perderías la transición que ya introduce el siguiente estado. La recuperación correcta consiste en partir de la carpeta de la práctica, aplicar de nuevo sus pasos numerados y observar la respuesta del navegador.

Fuentes de consulta

- Microsoft Learn: Fundamentos de Blazor.
- Microsoft Learn: Modos de renderizado.
- Microsoft Learn: Componentes, parámetros y composición.
- Microsoft Learn: Eventos y EventCallback.
- Microsoft Learn: Ciclo de vida de los componentes.